

Blockchain-Verified Distributed Storage: A DHT-Based Architecture with Cryptographic Proof of Retention

Craig Wright

University of Exeter, Exeter, United Kingdom

cw881@exeter.ac.uk

ORCID: 0000-0001-9374-0507

Abstract

Distributed data storage systems face a fundamental verification problem: how can a data owner confirm that a remote storage provider has retained the data it was paid to store, without downloading the entire dataset? Existing approaches require centralised trust authorities, impose prohibitive bandwidth costs, or lack enforceable economic consequences for storage failure. This paper proposes a blockchain-verified distributed storage architecture addressing these limitations through three integrated mechanisms. First, a distributed hash table (DHT) overlay partitions data across storage nodes using cryptographic key-addressed routing, reducing per-node storage from $O(D)$ to $O(D/N)$. Second, a Merkle tree proof-of-retention protocol enables any storage node to demonstrate continued possession of its assigned data blocks by producing compact cryptographic proofs—hash paths from challenged leaf nodes to a publicly committed root—without revealing the underlying data. Third, these proofs are embedded in blockchain transactions using a locking-script challenge mechanism, where a service oracle constructs transactions that can only be unlocked by a storage provider possessing the challenged data block and the corresponding Merkle proof. The architecture supports encrypted multi-copy storage with per-copy key differentiation for collusion resistance, RAID-inspired data striping for parallel retrieval, and reputation-scored admission for Sybil resistance. Monte Carlo simulation validation (10,000 trials per configuration) confirms analytical detection bounds within 2.2% and timing-based outsourcing detection achieving 99.65% statistical power with as few as five challenge epochs. For a 1 TB dataset across 100 nodes with replication factor 2, per-node storage is approximately 20 GB and Merkle proof size is $O(\log D)$ regardless of dataset size.

Keywords: distributed storage, blockchain verification, distributed hash table, Merkle tree, proof of retention, cryptographic audit, data integrity

1. Introduction

The proliferation of digital data—driven by cloud computing, Internet of Things telemetry, machine learning datasets, and multimedia content—has created storage demands that increasingly exceed the capacity and economic viability of centralised infrastructure. Global data creation is projected to reach 181 zettabytes by 2025 [1], yet the cost structures of hyperscale cloud providers are not independently verifiable, their pricing models are not subject to external audit, and their service guarantees are ultimately backed by contractual promises and reputational incentives rather than cryptographic proof. When a client pays a major cloud provider to store data, the client receives no cryptographic proof that the data has been retained and no mechanism to verify storage independently of the provider.

This verification gap is not merely theoretical. Cloud storage failures and silent data corruption have been documented in large-scale deployments [2,3], and de-duplication practices are typically not disclosed to clients. The fundamental problem is one of trust asymmetry: the storage provider possesses both the data and the authority to report on its status, creating an inherent conflict of interest. The client must trust the provider's self-report, because independent verification would require downloading and comparing the entire dataset—defeating the purpose of remote storage. Recent empirical studies have reinforced this concern: [4] demonstrated that multi-cloud environments amplify the integrity problem by introducing additional trust boundaries, while [5] showed that even blockchain-anchored IoT storage systems suffer from verification overhead that limits practical deployability.

Decentralised storage networks—including IPFS [6], Filecoin [7], Arweave [8], and Storj [9]—have emerged as alternatives that distribute data across independent storage providers. However, these systems face their own limitations. IPFS provides content-addressed retrieval but no native mechanism for verifying that content remains available at specified providers. Filecoin introduces proof-of-replication and proof-of-spacetime protocols, but these require computationally intensive zk-SNARK constructions that impose substantial hardware requirements on storage providers and introduce cryptographic complexity that limits accessibility. Filecoin's sealing process alone requires several hours of GPU computation per 32 GB sector, creating hardware barriers that may limit participation to well-resourced operators. Arweave uses a “blockweave” structure with a “succinct proof of random access” mechanism, but this is tightly coupled to its specific consensus model and cannot be applied to general-purpose blockchain networks. Storj distributes encrypted fragments across nodes but relies on satellite coordinators for verification, reintroducing centralisation. A comprehensive

survey by [10] confirmed that security, integrity, and privacy remain open challenges across these decentralised storage platforms.

This paper proposes an alternative approach: a distributed storage architecture that uses a standard public blockchain as the verification layer, requiring no specialised consensus mechanism, no zero-knowledge proofs, and no trusted coordinators. The core insight is that the data structures already present in blockchain transaction scripting—specifically, the ability to embed hash commitments in locking scripts and require Merkle proofs in unlocking scripts—provide a complete, general-purpose framework for verifiable storage. A service provider (oracle) commits a Merkle root derived from the stored data to the blockchain via a locking script. To demonstrate continued possession, a storage provider must produce an unlocking script containing a valid Merkle proof for a randomly challenged leaf node. If the storage provider has retained the data, producing this proof is trivial; if not, forging the proof requires breaking the collision resistance of the hash function—a computational impossibility under standard cryptographic assumptions.

The architecture integrates this Merkle proof-of-retention mechanism with a DHT-based data distribution layer that partitions data across storage nodes using cryptographic key-addressed routing, reducing per-node storage while maintaining $O(\log N)$ retrieval. Storage nodes are organised in an overlay network with multicast-based communication and reputation-scored admission to resist Sybil attacks. Encrypted multi-copy storage with per-copy key differentiation prevents collusion among storage providers, while RAID-inspired data striping enables parallel retrieval for performance optimisation.

The paper makes the following contributions. First, we propose a complete architecture for blockchain-verified distributed storage that combines DHT-based data partitioning with Merkle proof-of-retention enforced through blockchain transaction scripting. Second, we develop the cryptographic protocols for challenge generation, proof construction, and verification, including mechanisms for collusion resistance through encrypted multi-copy storage. Third, we present formal models of storage cost, proof complexity, collusion resistance, DHT availability, and routing latency. Fourth, we analyse the security properties of the system under various adversarial models including data deletion, proof forgery, Sybil attacks, and service provider compromise. Fifth, we discuss the extension of the architecture to RAID-inspired configurations and race-based proof mechanisms that introduce competitive incentives for high-availability storage.

The architecture we propose differs from existing systems in several fundamental respects. Unlike Filecoin, which requires a custom blockchain, custom consensus mechanism, and GPU-intensive zero-knowledge proof generation, our system operates on any existing UTXO-based blockchain with standard scripting capabilities. Unlike Storj, which relies on centralised satellite coordinators for verification, our system embeds verification in the blockchain's own transaction validation pipeline, requiring no trusted intermediaries. Unlike IPFS, which provides content addressing without storage verification, our system produces cryptographically verifiable, economically enforceable, and publicly auditable proof that data has been retained.

The economic dimension of the design is equally important. By embedding storage fees in the output of challenge transactions, the system creates a direct, atomic link between proof of retention and payment. A storage provider is paid if and only if it produces a valid proof; the blockchain enforces this conditionality without any party needing to trust any other. This eliminates the “trust then verify” model of conventional cloud storage and replaces it with a “verify then pay” model where verification is a precondition of payment, cryptographically enforced by the consensus network.

The remainder of the paper is organised as follows. Section 2 reviews related work across distributed storage, proof-of-storage protocols, DHT systems, and blockchain scripting. Section 3 presents the network architecture. Section 4 describes the Merkle proof-of-retention protocol, including a concrete BSV script implementation and privacy-preserving proof variants. Section 5 develops encrypted multi-copy storage for collusion resistance. Section 6 presents the formal analysis of storage, routing, and availability. Section 7 discusses RAID configurations and race-based proofs. Section 8 provides the formal security model with reduction-style proofs of possession soundness, outsourcing resistance, and selective-retention detection. Section 9 presents the systems evaluation, including on-chain cost models, economic sustainability analysis, and DHT churn resilience. Section 10 discusses implications, comparison with existing systems, and limitations. Section 11 concludes.

2. Related Work

2.1. Distributed Storage Systems

Distributed storage systems distribute data across multiple nodes to achieve fault tolerance, scalability, and geographic availability. Google's GFS [11] and its open-source successor HDFS [12] introduced the paradigm of distributing large files as fixed-size chunks across commodity hardware, with a centralised master server managing metadata and chunk

placement. Amazon's Dynamo [13] extended the model to a fully decentralised key-value store using consistent hashing for data placement, vector clocks for conflict resolution, and configurable replication for availability. These systems demonstrated that distributed storage at scale is feasible but remained within the trust boundary of a single organisation. No mechanism exists for external parties to verify that data is retained as promised.

Peer-to-peer storage systems removed the organisational trust boundary. OceanStore [14] proposed a global persistent storage system using Byzantine fault-tolerant replication and inner-ring replication for high availability. CFS [15] built a read-only cooperative file system atop the Chord DHT. These systems addressed data distribution but lacked cryptographic verification of continued storage. A dishonest node could claim to store data while having deleted it, and the system relied on redundancy rather than verification to ensure availability. More recently, [5] proposed FileInsurer, a protocol combining blockchain with IPFS for scalable decentralised file storage, but their approach relies on Ethereum smart contracts rather than native transaction scripting and incurs gas costs proportional to verification frequency. [16] devised a decentralised storage framework merging Hyperledger Fabric with IPFS and derived a mathematical formula to quantify file security based on allocation functions, though their framework remains constrained to permissioned blockchain environments.

2.2. Proof-of-Storage Protocols

The theoretical foundations for verifiable storage were established by provable data possession (PDP) protocols [17] and proof-of-retrievability (POR) schemes [18]. PDP enables a client to verify that a remote server possesses a file without downloading it, using homomorphic verifiable tags attached to file blocks. POR extends this to guarantee that the file can actually be retrieved. [19] improved POR with compact proofs based on BLS signatures. These protocols operate in a two-party model (client and server) and require the client to perform verification. They do not natively support public verifiability—the ability for any third party to check the proof without holding a secret key. [20] introduced public auditing for cloud storage using bilinear maps, enabling third-party auditors to verify data integrity. However, the auditor remains a trusted entity, and the verification results are not publicly committed to an immutable ledger.

Recent work has extended these foundations in several directions. [21] proposed a hierarchical Merkle tree structure with local signature aggregation to reduce tree scale and improve update efficiency for dynamic cloud data auditing. [4] developed a collaborative integrity auditing scheme (CIA) for multi-replica cloud storage that addresses the multi-provider verification

problem but relies on a trusted third-party auditor. [22] introduced a quad Merkle tree structure with blockchain anchoring that reduces hash computation overhead compared to binary trees while committing audit results to an immutable ledger. [23] proposed an improved Merkle Mountain Range for IoT data storage verification, achieving sampling-based integrity checks that reduce network overhead. [24] provided a rigorous analysis of data falsification probabilities in Merkle trees, establishing theoretical bounds on collision resistance as functions of hash length and tree depth—results that directly inform the security guarantees of the proof-of-retention protocol proposed here.

Blockchain-based storage verification has advanced through several platforms. Filecoin [7] introduced proof-of-replication (PoRep) and proof-of-spacetime (PoSt), both verified using zk-SNARKs. However, Filecoin's sealing process requires several hours of GPU computation per 32 GB sector, and the WindowPoSt mechanism requires storage providers to respond within a 30-minute deadline with zk-SNARK proofs—imposing substantial hardware requirements [25]. Sia [26] uses Merkle-based storage proofs in its custom blockchain, but requires its own cryptocurrency and consensus mechanism. None of these systems leverage the scripting capabilities of general-purpose UTXO-based blockchains to embed storage verification directly in standard transactions, which is the approach taken in the present work.

2.3. Distributed Hash Tables

Distributed hash tables provide the algorithmic foundation for partitioning data across a peer-to-peer network. Chord [27] uses consistent hashing in a circular identifier space, achieving $O(\log N)$ lookup with a routing table of $O(\log N)$ entries. Pastry [28] assigns each node a 128-bit identifier and routes by prefix matching in $O(\log_{16} N)$ hops using three complementary data structures: a routing table for long-distance forwarding, a neighbourhood set for proximity-aware routing, and a leaf set for keyspace-neighbour resolution. Kademlia [29] introduces an XOR-based distance metric providing symmetric routing and preferential treatment of long-lived nodes. CAN [30] maps keys to a d-dimensional Cartesian space. Sybil-resistant DHTs have been proposed using social-trust graphs [31], proof-of-work puzzles [32], and reputation systems [33]. [34] demonstrated that Sybil resistance can be combined with certificateless cryptography to achieve data integrity verification in cloud-edge environments with sparse Merkle trees, reducing communication overhead by 75% through pre-download mechanisms.

2.4. Blockchain Transaction Scripting

UTXO-based blockchain protocols support expressive scripting in transaction outputs (locking scripts) and inputs (unlocking scripts). The locking script defines conditions for spending; the

unlocking script provides satisfying data. Standard use cases include signature verification, multi-signature requirements, and time-locked outputs. The scripting language supports arbitrary hash-based conditions: a locking script can embed a hash value and require the unlocking script to provide a matching pre-image. This capability has been used for hash time-locked contracts in payment channels [35] and for atomic swaps across different blockchain networks, but has not been systematically applied to storage verification.

The key insight of the present work is that Merkle proof verification can be expressed entirely within the blockchain scripting language. A locking script can embed a Merkle root and require the unlocking script to provide a leaf pre-image and a sequence of sibling hashes constituting a valid Merkle proof. The script iteratively hashes pairs of values up the tree and verifies that the final hash equals the embedded root. This embeds storage verification directly in the consensus layer: a response transaction is valid if and only if it contains a correct Merkle proof, and its validity is verified by every node in the network as part of standard transaction validation. This approach transforms every mining node into a storage auditor at zero marginal cost, requiring no additional protocol changes, trusted parties, or specialised hardware.

3. Network Architecture

3.1. System Model

The system comprises three classes of participants. **Data providers** (clients) possess data items to be stored remotely. **Storage providers** commit storage resources and receive compensation for retaining data. A **service provider** (oracle) mediates between data providers and storage providers, managing data preparation, Merkle tree construction, challenge generation, and proof verification. In some configurations, the data provider and service provider may be the same entity. All participants interact through the blockchain network, which provides the immutable, publicly auditable ledger for challenge-response transactions. Figure 1 illustrates the overall system architecture and the relationships between these participants.

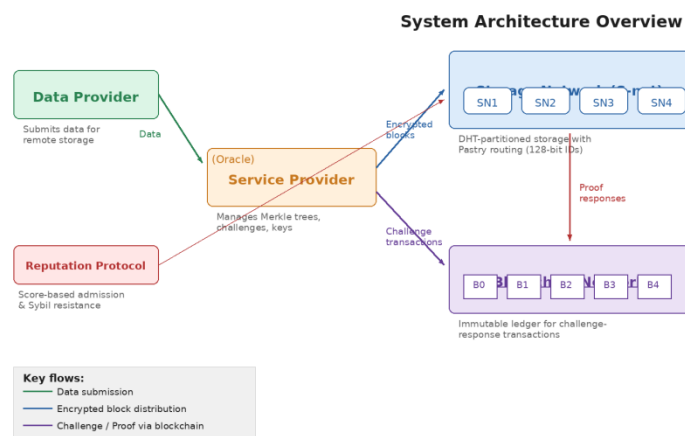


Figure 1. System architecture showing three participant classes (data provider, service provider, storage providers) interacting through the blockchain network. Storage nodes form a DHT overlay (S-net) for partitioned data storage, while the blockchain provides the immutable verification layer for challenge-response transactions.

We define the storage network as a graph $G = (V, E)$ where $V = V_s \cup V_r$ consists of storage nodes V_s that store data and produce retention proofs, and regular blockchain nodes V_r that perform mining, consensus, and transaction validation. Storage nodes form an overlay sub-network (the S-net) dedicated to data storage and retrieval. They do not mine blocks or maintain the complete blockchain; their function is to store assigned data partitions, respond to retention challenges, and participate in DHT routing for data retrieval. Storage nodes connect to both the S-net overlay (for data operations) and the regular blockchain network (for submitting and monitoring challenge-response transactions). This two-tier architecture mirrors the separation of concerns advocated by recent work on overlay-based blockchain scalability, where specialised node classes perform dedicated functions without modifying the consensus layer.

3.2. DHT-Based Data Distribution

When a data provider submits a data item for storage, the service provider divides it into a sequence of fixed-size data blocks. Each data block is assigned a cryptographic key by hashing its content or index, and the DHT routing protocol determines which storage node(s) are responsible for that key. The data block is then transmitted to the responsible storage node(s) via a $\text{put}(k, \text{block})$ operation. This partitioning ensures that no single storage node holds the

complete dataset, distributing both the storage burden and the risk of data loss across the network.

We adopt the Pastry DHT protocol. Each storage node is assigned a 128-bit identifier uniformly at random in $[0, 2^{128})$. Each node maintains three routing structures. The *routing table* maps key prefixes to node addresses, enabling long-distance forwarding by matching progressively longer prefixes. The *neighbourhood set* tracks physically proximate peers by network latency, optimising local routing decisions. The *leaf set* contains the $L/2$ nodes with numerically closest identifiers on each side, enabling direct routing to keyspace neighbours. Together these guarantee that any key can be resolved in at most $\lceil \log_{16} |V_s| \rceil$ forwarding steps. Figure 3 illustrates the DHT routing structure and per-node data structures.

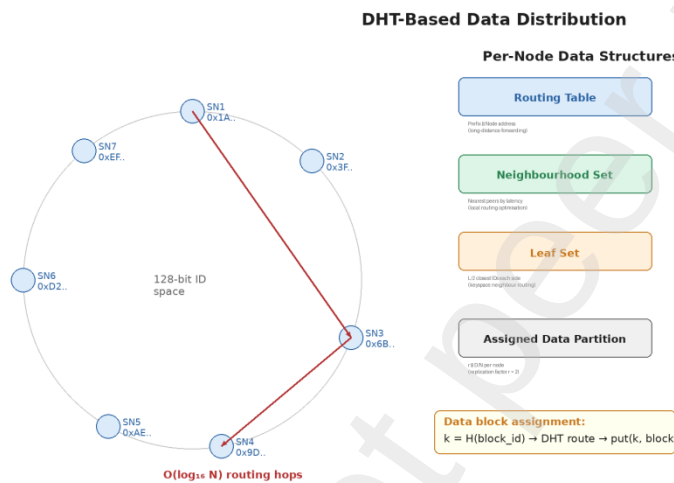


Figure 3. DHT-based data distribution. Left: Pastry circular identifier space with storage nodes and $O(\log N)$ routing path. Right: per-node data structures comprising routing table, neighbourhood set, leaf set, and assigned data partition.

A configurable replication factor r (default $r = 2$) ensures that each data block is stored at r storage nodes with adjacent DHT identifiers. Per-node storage is $O(r \times D / |V_s|)$, where D is the total data volume. For a 1 TB dataset distributed across 100 storage nodes with $r = 2$, each node stores approximately 20 GB—a $50\times$ reduction compared to full replication, where each node would store the entire 1 TB.

3.3. Overlay Communication

The storage overlay (S-net) employs IP multicast for efficient communication among storage nodes. When a storage node receives new data blocks for distribution, it transmits them to a multicast group address to which all S-net nodes subscribe, providing $O(1)$ fan-out from the

sender's perspective. This eliminates the per-peer connection establishment overhead of unicast communication, which scales linearly with the number of recipients.

For inter-node data transfer, storage nodes use UDP rather than TCP, eliminating the three-way handshake (consuming 160–296 bytes and one round-trip time per connection) and avoiding head-of-line blocking under packet loss. Application-level reliability—acknowledgement tracking, selective retransmission, and duplicate detection—operates above the transport layer, providing the reliability guarantees needed for data transfer without imposing TCP's ordering and flow-control overhead. Regular blockchain nodes continue to use TCP via the standard protocol. The S-net overlay operates alongside, not instead of, the existing peer-to-peer network, ensuring that no changes to the base-layer communication protocol are required.

3.4. Storage Node Internal Architecture

Each storage node maintains the following components in memory: the storage node application software; the assigned fraction of the DHT keyspace (routing table, neighbourhood set, leaf set); the stored data blocks for its assigned partition; and a *reputation table* for its neighbourhood. The reputation table records, for each neighbour: a node identifier, a join timestamp, and a behaviour score. The score is initialised at a neutral value upon joining and adjusted based on observable behaviours: prompt forwarding of DHT queries (positive), consistent responsiveness (positive), extended silence (negative), and flooding of non-transactional traffic (negative). When a neighbour's score falls below a configurable threshold, the monitoring node designates it as malicious and severs the connection. This distributed reputation mechanism provides Sybil resistance without central authority, operating on the assumption that honest nodes form the majority and that adversarial behaviour is locally observable by DHT neighbours.

3.5. Node Admission and Sybil Resistance

A new node joins the S-net by submitting an admission request to an existing storage node, which validates the applicant's hardware resources (memory, storage capacity, network bandwidth) and forwards the request to the DHT for identifier assignment and routing table provisioning. Upon admission, the new node receives its routing table, leaf set, neighbourhood set, and DHT key assignment from the existing network. The reputation score starts at a neutral value, requiring the new node to demonstrate sustained positive behaviour before it gains full trust within its neighbourhood.

This admission protocol converts a Sybil attack from a one-time identity creation cost into a sustained resource expenditure. An attacker controlling f Sybil identities must invest $f \times t_{\text{build}}$ cumulative time before those identities can influence routing or suppress transactions, where t_{build} is the time required to reach a trusted reputation score. Furthermore, eviction decisions are made independently by each monitoring node: when multiple neighbours independently lower a node's score below threshold, the offending node loses connectivity to the S-net without any single point of failure in the eviction mechanism. An attacker must simultaneously deceive all L leaf-set neighbours to avoid detection, requiring control of a neighbourhood-sized fraction of the network—substantially harder than merely creating identities.

3.6. DHT Adversary Model: Eclipse Attacks and Churn Manipulation

The Sybil resistance mechanisms in Section 3.5 address identity-creation attacks. We now analyse two additional classes of DHT-layer attacks: eclipse attacks (targeted routing table poisoning) and churn manipulation (strategic join/leave behaviour). Both exploit the structure of the DHT overlay rather than the identity system.

3.6.1. Eclipse attacks.

Definition 6 (Eclipse Attack). An eclipse attack on node v targets v 's routing state by populating v 's routing table, leaf set, and neighbourhood set with adversary-controlled nodes. If successful, the adversary mediates all of v 's DHT lookups and can: (a) suppress data retrieval requests, (b) intercept challenge-response traffic, and (c) partition v from the honest network.

Attack surface in Pastry. Under Pastry, a node v maintains three routing structures: a routing table with $O(\log_{16} N)$ rows of 16 entries each, a leaf set of L nodes (default $L = 16$), and a neighbourhood set of M nodes (default $M = 16$). To fully eclipse v , the adversary must control: (i) all $L/2$ left and $L/2$ right leaf-set neighbours of v in the identifier space, (ii) at least one entry per row of v 's routing table, and (iii) all M neighbourhood-set entries.

Resistance analysis. Let α denote the fraction of network nodes controlled by the adversary. Under uniform random identifier assignment (Section 3.5), the probability that all L leaf-set positions adjacent to v are adversary-controlled is α^L . For $L = 16$ and $\alpha = 0.2$ (20% adversarial control), this probability is $0.2^{16} \approx 6.6 \times 10^{-12}$. Even at $\alpha = 0.4$ (40% control), the eclipse probability for the leaf set alone is $0.4^{16} \approx 4.3 \times 10^{-7}$. Since eclipse requires simultaneous control of the leaf set, routing table, and neighbourhood set, the joint probability is strictly lower. Table 1 quantifies eclipse probability as a function of adversarial network fraction.

Table 1. Eclipse attack probability by adversarial network fraction ($N = 1,000$, $L = 16$)

α (adversary)	Leaf set (α^{16})	Route table*	Neighbour set	Joint eclipse	N nodes at risk†
0.10	10^{-16}	$\sim 10^{-4}$	10^{-16}	$< 10^{-36}$	< 1 in 10^{33}
0.20	6.6×10^{-12}	$\sim 10^{-2}$	6.6×10^{-12}	$< 10^{-26}$	< 1 in 10^{23}
0.30	4.3×10^{-9}	$\sim 10^{-1}$	4.3×10^{-9}	$< 10^{-19}$	< 1 in 10^{16}
0.40	4.3×10^{-7}	~ 0.3	4.3×10^{-7}	$< 10^{-13}$	< 1 in 10^{10}

Routing table eclipse requires adversary entry in all $\log_{16} N$ rows; computed for $N = 1,000$ (3 rows). †Expected number of honest nodes eclipsed in a 1,000-node network.

Routing table eclipse derivation. For $N = 1,000$ nodes, the Pastry routing table has $\lceil \log_{16} 1000 \rceil = 3$ rows. Each row has 16 slots, but only one entry per slot is maintained. The probability that all entries in a given row are adversary-controlled is approximately α^{16} per row (under uniform distribution of identifiers across slots). The probability that at least one adversary node occupies each row is $(1 - (1 - \alpha)^{16})^3$. For $\alpha = 0.2$, this is $(1 - 0.8^{16})^3 \approx (1 - 0.028)^3 \approx 0.92^3 \approx 0.78$. However, controlling one entry per row provides only routing influence, not full eclipse; full eclipse requires controlling all entries, yielding the probabilities in Table 1.

Region-targeted attacks. The independence assumption underlying Table 1 may not hold if the adversary targets a specific region of the identifier space. In a targeted attack, the adversary generates Sybil identities with identifiers concentrated near the target's position. Under Section 3.5's admission protocol, identifier assignment is performed by existing nodes using a verifiable random function (VRF): $\text{id} = \text{VRF}(\text{joining}_{\text{node}} \text{pubkey}, \text{nonce})$. The joining node cannot choose its identifier, and the assigning node's VRF output is publicly verifiable. This prevents identifier clustering. If the admission protocol instead uses self-selected identifiers (as in some Kademlia implementations), region targeting becomes feasible, and the independence assumptions in Table 1 no longer hold. We therefore specify VRF-based identifier assignment as a protocol requirement, not an optional feature.

3.6.2. Churn manipulation.

Definition 7 (Churn Manipulation Attack). A churn manipulation attack exploits the DHT's recovery mechanisms by strategically timing node joins and departures. The adversary alternates Sybil identities between active and inactive states to: (a) trigger unnecessary re-replication, consuming bandwidth and storage; (b) cause routing instability, increasing lookup latency; or (c) occupy replica positions during re-replication events, substituting adversary-held copies for honest replicas.

Analysis. Under the Pastry protocol, when a node v departs, its leaf-set neighbours detect the departure (via periodic heartbeat failure) and initiate re-replication of v 's assigned data blocks. If adversary node v' occupies the successor position in the identifier space, it receives the re-replicated data. However, v' is then subject to the same challenge-response protocol as any storage provider: it must produce valid Merkle proofs to earn compensation and avoid slashing.

The economic cost of churn manipulation is quantified as follows. Each Sybil cycle (join, build reputation over t_{build} , strategically depart, rejoin) incurs: (i) reputation-building time t_{build} , during which the adversary must behave honestly; (ii) resource commitment (storage capacity, bandwidth) proportional to the assigned DHT partition; and (iii) deposit D_{dep} locked during the retention contract. For $t_{\text{build}} = 30$ days and D_{dep} calibrated to 90 days of epoch payments, the adversary's capital cost per Sybil cycle is $90 \times \alpha \times f_{\text{epoch}}$ satoshis locked for at least 30 days. Sustaining f Sybil identities costs $f \times 90\alpha f_{\text{epoch}}$ in locked capital, creating a direct economic barrier proportional to attack scale.

3.6.3. Mitigation summary.

Table 2. DHT adversary model: attacks, defences, and residual risks

Attack	Mechanism	Primary Defence	Residual Risk
Sybil infiltration	Mass identity creation	Reputation building (t_{build}); deposit locking	Slow infiltration over months
Eclipse (targeted)	Routing table poisoning	VRF identifier assignment; leaf set size $L = 16$	Requires $\alpha > 0.4$ for non-negligible probability
Eclipse (region)	Identifier clustering	VRF-based admission (no self-selected IDs)	Eliminated under VRF requirement
Churn manipulation	Strategic join/leave	Deposit lock; t_{build} delay; re-replication monitoring	Capital cost proportional to attack scale
Data suppression	Withhold stored blocks	Replication $r \geq 2$; challenge-response detection	Requires control of all r replicas

4. Merkle Proof-of-Retention Protocol

4.1. Data Preparation and Merkle Tree Construction

The service provider divides a data item Δ into n fixed-size blocks: $\Delta = (b_1, b_2, \dots, b_n)$. For each block, a leaf hash is computed: $\text{leaf}_i = H(b_i)$, where H is a collision-resistant hash function (SHA-256). These leaf hashes form the bottom level of a binary Merkle tree. Internal nodes are computed by concatenating and hashing pairs of children: $h_{\text{parent}} = H(h_{\text{left}} \parallel h_{\text{right}})$. The process repeats until a single root r_M is obtained. The root is committed to the blockchain by embedding it in a locking script.

Challenge generation. At each challenge epoch, the service provider selects a random block index i and constructs a challenge transaction T_{chal} . The locking script of T_{chal} embeds the Merkle root r_M , the challenged index i , and the Merkle proof verification logic as a sequence of opcodes.

4.2. Challenge-Response via Blockchain Transactions

Response construction. The storage provider, holding block b_i , constructs a response transaction T_{resp} that spends the output of T_{chal} . The unlocking script of T_{resp} provides: (a) the data block b_i (or its hash pre-image), and (b) the sequence of sibling hashes along the Merkle path from leaf i to the root. For a tree of depth $d = \lceil \log_2 n \rceil$, this requires d sibling hashes, totalling $d \times 32$ bytes for SHA-256.

On-chain verification. When T_{resp} is submitted, every validating node executes the locking and unlocking scripts together as standard transaction validation. The script iteratively hashes pairs up the tree and compares to the embedded root. Valid transactions are included in the blockchain; invalid ones are rejected. This requires no specialised consensus rules, no additional message types, and no changes to the mining protocol. Figure 2 illustrates the complete challenge-response process and Merkle tree structure.

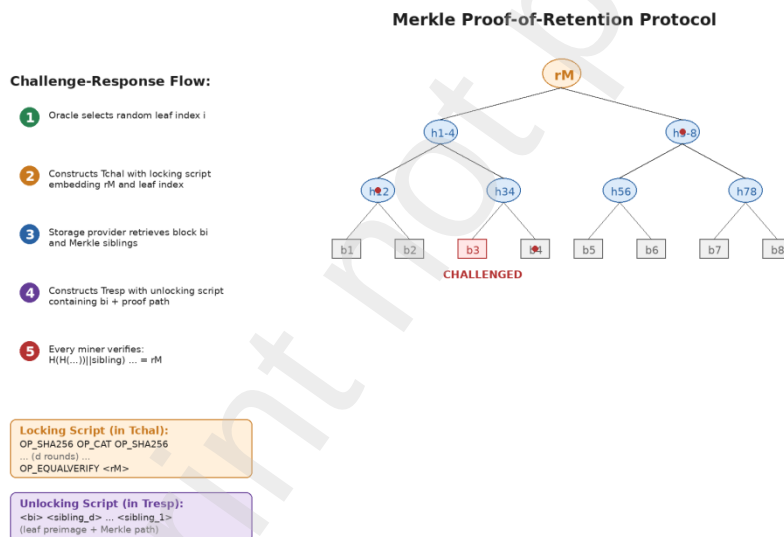


Figure 2. Merkle proof-of-retention protocol. Left: five-step challenge-response flow from oracle challenge generation through on-chain verification. Right: Merkle tree structure

showing challenged leaf node (b_3), proof path (highlighted siblings), and root verification. Bottom: locking and unlocking script structures.

4.3. Locking Script Construction

The locking script encodes the Merkle root and verification logic. For tree depth $d = \lceil \log_2 n \rceil$, the script performs d rounds of hash-concatenate-hash, then compares to the embedded root using standard opcodes: OP_SHA256, OP_CAT, OP_SWAP, OP_EQUALVERIFY. For $n = 8$ blocks (depth 3), challenging block b_3 requires the unlocking script to provide b_3 and three sibling hashes (h_4, h_{12}, h_{5-8}). Script execution: (1) hash $b_3 \rightarrow h_3$, (2) concatenate $h_3||h_4$ and hash $\rightarrow h_{34}$, (3) concatenate $h_{12}||h_{34}$ and hash $\rightarrow h_{1-4}$, (4) concatenate $h_{1-4}||h_{5-8}$ and hash $\rightarrow$ candidate root, (5) OP_EQUALVERIFY against r_M . For the elliptic curve method, OP_CHECKSIG is added to verify the ECDSA signature, providing privacy-preserving verification without revealing the underlying data.

4.4. Periodic Challenge Scheduling

Challenges are issued at intervals τ determined by the storage agreement. At each epoch, the service provider selects a random leaf index. The storage provider must respond before the next epoch. Random selection ensures probabilistic coverage: after k epochs, the probability a specific block has been challenged at least once is $1 - (1 - 1/n)^k$. For $n = 1000$ and $k = 7000$, this exceeds 0.999. The challenge schedule and response lifecycle are illustrated in Figure 5.

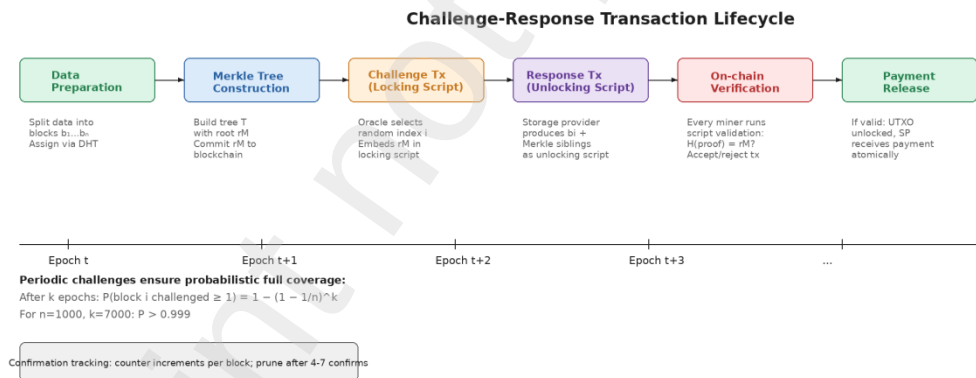


Figure 5. Challenge-response transaction lifecycle showing the six-stage process from data preparation through payment release, with periodic epoch scheduling ensuring probabilistic coverage of all stored blocks over time.

The random index selection can be made publicly verifiable by deriving it from a deterministic function of the most recent block hash, preventing the service provider from selectively avoiding challenges to compromised storage providers. This transforms challenge selection

from a trusted operation into a verifiable computation anchored in the blockchain's own randomness source, eliminating the last point of trust in the challenge generation process.

4.5. Concrete Script Implementation: BSV as Reference Chain

While the proof-of-retention protocol is expressible on any UTXO chain with sufficient scripting capability, we provide a concrete, fully specified implementation targeting the Bitcoin SV (BSV) blockchain. BSV is selected as the reference chain because its Genesis upgrade [37] removed all script size limits, opcode count limits, and stack element size constraints that constrain scripting on other UTXO chains. This makes BSV the natural target for on-chain Merkle proof verification. We emphasise, however, that the protocol can be ported to any UTXO chain whose scripting language supports SHA-256 hashing (OP_SHA256), concatenation (OP_CAT), conditional branching (OP_IF/OP_NOTIF/OP_ENDIF), stack manipulation (OP_SWAP), and equality testing (OP_EQUAL). The requirement is functional, not platform-specific.

Unlocking script (scriptSig). The storage provider pushes the following data onto the stack in order (first pushed = deepest in stack): the d direction bits and d sibling hashes in reverse level order, then the challenged block data. After all pushes complete, the stack (top to bottom) reads:

$b_i, \text{dir}_1, s_1, \text{dir}_2, s_2, \dots, \text{dir}_d, s_d$

where b_i is the challenged data block, $\text{dir}_j \in \{0, 1\}$ indicates whether the current hash is the left (0) or right (1) child at level j , and s_j is the sibling hash at level j .

Locking script (scriptPubKey). The locking script performs iterative Merkle path verification using exactly $6d + 3$ opcodes for a tree of depth d :

OP_SHA256 // hash $b_i \rightarrow h$

// Repeat for each level $j = 1, \dots, d$:

OP_SWAP // bring dir to top

OP_NOTIF // if dir = 0 (current hash is left child)

OP_SWAP // swap so h is below sibling

OP_ENDIF

OP_CAT // concatenate (left || right)

OP_SHA256 // hash pair $\rightarrow$ parent

// End repeat

$\langle r_M \rangle$ OP_EQUAL // compare to committed Merkle root

The per-level block (OP_SWAP, OP_NOTIF, OP_SWAP, OP_ENDIF, OP_CAT, OP_SHA256) comprises exactly 6 opcodes. The OP_CAT operation in BSV script concatenates the second-to-top element with the top element, so when $dir_j = 0$ (current hash is the left child), the OP_SWAP inside the NOTIF branch reorders the stack so that h is placed below the sibling s_j , producing the concatenation $h \parallel s_j$ (left || right). When $dir_j = 1$ (current hash is the right child), no swap occurs, and OP_CAT produces $s_j \parallel h$ (left || right). In both cases, the correct parent hash is computed.

Execution trace for depth 3 ($n = 8$ blocks). Challenging block b_3 (index 2 in zero-based numbering, binary path 010):

Table 3. BSV script execution trace for depth-3 Merkle proof (challenging b_3)

Step	Opcode	Stack (top $\rightarrow$ bottom)	Effect
0	(scriptSig)	$b_3, 0, s_1, 1, s_2, 0, s_3$	Push proof data
1	OP_SHA256	$h_3, 0, s_1, 1, s_2, 0, s_3$	Hash leaf
2	OP_SWAP	$0, h_3, s_1, 1, s_2, 0, s_3$	Bring dir_1
3	OP_NOTIF	$h_3, s_1, 1, s_2, 0, s_3$	$dir=0$: enter body
4	OP_SWAP	$s_1, h_3, 1, s_2, 0, s_3$	Reorder for left child
5	OP_CAT	$h_3 \parallel s_1, 1, s_2, 0, s_3$	Concatenate L R
6	OP_SHA256	$h_{34}, 1, s_2, 0, s_3$	Hash $\rightarrow$ parent
7–12	(repeat)	$h_{1-4}, 0, s_3$	Levels 2–3
13	OP_SHA256	h_{root}	Final hash
14	$\langle r_m \rangle$ OP_EQUAL	TRUE/FALSE	Verify root

Script and transaction size analysis. The following bounds hold for a Merkle tree of depth d :

Table 4. Transaction size and cost analysis for BSV Merkle proof verification

Component	Formula	$d = 10$	$d = 20$	$d = 30$
Locking script	$6d + 3 + 32$ bytes	95 B	155 B	215 B
Unlocking script	$32 + 33d$ bytes	362 B	692 B	1,022 B
Total proof tx overhead	$33d + 67$ bytes	~457 B	~847 B	~1,237 B
Opcodes executed	$6d + 3$	63	123	183
SHA-256 operations	$d + 1$	11	21	31

BSV tx fee (1 sat/byte)	~500 sat	~900 sat	~1,300 sat	
USD at \$50/BSV	~\$0.00025	~\$0.00045	~\$0.00065	

A depth-30 Merkle proof (covering approximately 10^9 data blocks, i.e. a multi-terabyte dataset at 1 KB block size) produces a proof transaction of approximately 1,237 bytes, requiring 183 opcodes and 31 SHA-256 operations. On BSV, where no opcode or script size limits apply post-Genesis, this executes within standard validation. The full challenge-response cycle at $d = 30$ totals approximately 1,452 bytes. At BSV's current fee rate (approximately 1 satoshi per byte) the cycle cost is approximately 0.00073 USD. For 100 storage providers challenged daily, the annual on-chain cost is approximately $100 \times 365 \times \$0.00073 \approx \27 , confirming the economic viability of the verification mechanism at scale.

Portability to other UTXO chains. The script template above uses only five opcodes: OP_SHA256, OP_CAT, OP_SWAP, OP_NOTIF/OP_ENDIF, and OP_EQUAL. Any UTXO chain supporting these (or equivalents) can host the protocol. On chains with script size limits (e.g., the BTC network limits scripts to 10,000 bytes and 201 non-push opcodes), depth is constrained: the 201-opcode limit permits $d \leq 33$, covering datasets up to $2^{33} \approx 8.6$ billion blocks. On chains without such limits (BSV, BCH with OP_CAT restored), depth is unbounded in principle. The architecture is thus chain-agnostic in design, with BSV providing the most permissive execution environment.

4.6. Privacy-Preserving Proofs

A critical implementation concern is that the unlocking script reveals the challenged block b_i on-chain, creating a permanent data leak if plaintext blocks are used. We address this through three mechanisms, each appropriate for different threat models.

Ciphertext-only proofs. In the encrypted multi-copy scheme (Section 5), each storage provider holds encrypted blocks $c_{j,i} = b_i \oplus K_{j,i}$. The Merkle tree is constructed over ciphertext, so $\text{leaf}_{j,i} = H(c_{j,i})$. The unlocking script reveals $c_{j,i}$, not b_i . Since the storage provider does not possess $K_{j,i}$, it cannot derive the plaintext, and on-chain observers see only ciphertext. Under IND-CPA security of the encryption scheme, the ciphertext reveals no information about the plaintext.

Hash-only proofs. An alternative avoids revealing even ciphertext on-chain. The locking script embeds both the Merkle root and a specific leaf hash $h_i = H(b_i)$. The unlocking script provides only the sibling hashes for the Merkle path. The script hashes the embedded h_i upward through the tree and verifies against the root. This proves that h_i is in the committed tree, but requires

the storage provider to additionally prove knowledge of the preimage of h_i via a separate OP_SHA256 check, without publishing the preimage itself. This can be achieved by requiring the provider to sign a message with a key derived from the preimage, verified via OP_CHECKSIG.

EC-point proofs (privacy-preserving). The strongest privacy guarantee uses the EC method from Section 5.3. $\text{Leaf}_{j,i} = H(c_{j,i} \times G)$, and the unlocking script contains an ECDSA signature verifiable with public key $P_{j,i} = c_{j,i} \times G$ plus the Merkle sibling path. The on-chain script verifies: (a) the signature is valid under $P_{j,i}$, proving knowledge of $c_{j,i}$ without revealing it, and (b) $H(P_{j,i})$ hashes correctly through the Merkle tree to the root. In BSV script, this combines OP_CHECKSIGVERIFY with the iterative Merkle verification above, adding exactly one opcode to the template. No data block, ciphertext, or private key is ever published on-chain.

5. Encrypted Multi-Copy Storage for Collusion Resistance

5.1. The Collusion Problem

When multiple storage providers store copies of the same data, they may collude: rather than each storing an independent copy, they share one copy and split fees. Since each receives the same blocks and constructs the same Merkle tree, they can share proof responses without detection. This reduces actual redundancy below the contracted level, creating a silent failure mode where the system reports higher redundancy than actually exists.

5.2. Per-Copy Encryption

We resolve collusion by encrypting each copy with a different key. The service provider generates m distinct keys $K_1, \dots, K_m$ (one per storage provider) and produces distinct ciphertexts: $C_j = \text{Enc}(K_j, \Delta)$ for $j = 1, \dots, m$. Using AES-CTR mode, $c_{j,i} = b_i \oplus K_{j,i}$. Each provider SP_j receives only C_j . The service provider constructs a separate Merkle tree T_j for each ciphertext with root $r_{M,j}$. Challenges for SP_j embed $r_{M,j}$, so a valid proof for SP_j 's tree cannot be generated from SP_k 's data ($j \neq k$). Storage providers lack decryption keys and cannot transform their ciphertext to match another's tree. Figure 4 illustrates the encrypted multi-copy architecture.

Encrypted Multi-Copy Collision Resistance

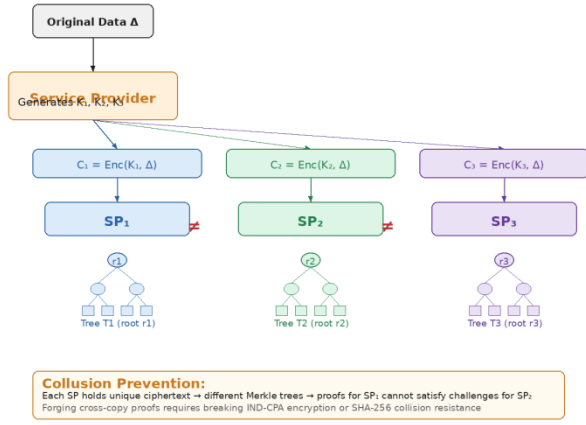


Figure 4. Encrypted multi-copy storage for collusion resistance. The service provider generates distinct encryption keys K_1, K_2, K_3 for each storage provider, producing unique ciphertexts and independent Merkle trees. Proofs for one provider cannot satisfy challenges for another.

5.3. EC Method for Privacy-Preserving Multi-Copy Proofs

Combined with elliptic curve (EC) point leaves, per-copy encryption provides collusion resistance and privacy preservation simultaneously. For encrypted copy C_j , $\text{leaf}_{j,i} = H(c_{j,i} \times G) = H(P_{j,i})$, where G is the generator point of the elliptic curve group. The storage proof requires SP_j to produce an ECDSA signature verifiable with $P_{j,i}$ as public key, without revealing $c_{j,i}$. This ensures: (a) ciphertext blocks are never exposed on-chain, (b) each provider's proof is unique and non-transferable, and (c) the service provider verifies possession without the storage provider learning the original plaintext data.

5.4. Optimised Cross-Tree Proof Reuse

A more efficient variant exists when the service provider first encrypts once with a master key to produce C , then creates per-provider variants by adding distinct secret values: $c_{j,i} = c_i + s_{j,i} \pmod{p}$. Leaves are $\text{leaf}_{j,i} = H((c_i + s_{j,i}) \times G)$. Given a proof for one provider, the service provider can verify the corresponding leaf for another by computing $P_{k,i} = P_{j,i} + (s_{k,i} - s_{j,i}) \times G$, since it knows all secret values. This enables Merkle proof reuse across trees, reducing the number of independent proofs required for multi-copy verification from m to 1, with computational overhead of $m - 1$ elliptic curve point additions.

5.5. Security Analysis of the EC-Point Proof Variant

We formalise the security of the elliptic curve (EC) point proof variant introduced in Sections 4.6 and 5.3. The construction combines Merkle tree commitment with ECDSA signature

verification and requires formal treatment of three properties: proof soundness, zero-knowledge (no data leakage), and binding (non-transferability).

Cryptographic assumptions.

Assumption 1 (Elliptic Curve Discrete Logarithm Problem, ECDLP). For a cyclic group $\langle G \rangle$ of prime order q on an elliptic curve E , no probabilistic polynomial-time algorithm can compute x from $P = x \times G$ with non-negligible probability.

Assumption 2 (ECDSA Existential Unforgeability, EU-CMA). The ECDSA signature scheme is existentially unforgeable under chosen-message attack. That is, no PPT adversary with access to a signing oracle can produce a valid signature on a new message with non-negligible probability.

Assumption 3 (Collision Resistance of H). As in Theorem 2, $H: \{0,1\}^* \rightarrow \{0,1\}^{256}$ is (t, ϵ_1) -collision-resistant.

Construction recap. For encrypted copy C_j assigned to storage provider SP_j , each ciphertext block $c_{j,i}$ is treated as a scalar in $\mathbb{Z}_q$. The EC-point leaf is computed as $\text{leaf}_{j,i} = H(c_{j,i} \times G) = H(P_{j,i})$, where G is the generator of the elliptic curve group and $P_{j,i}$ is the corresponding public key. The Merkle tree T_j is constructed over the set $\{\text{leaf}_{j,i}\}$ for $i = 1$ to n . To respond to a challenge on block i , SP_j produces: (a) an ECDSA signature σ on a canonical challenge message m (e.g., the challenge transaction hash) using $c_{j,i}$ as the private key, and (b) the Merkle sibling path $(s_1, \dots, s_d)$ from $\text{leaf}_{j,i}$ to root $r_{M,j}$. The locking script verifies σ against $P_{j,i}$ via `OP_CHECKSIGVERIFY`, then verifies that $H(P_{j,i})$ hashes through the sibling path to $r_{M,j}$.

Theorem 8 (EC-Point Proof Soundness). *If H is (t, ϵ_1) -collision-resistant and ECDSA is (t, ϵ_2) -EU-CMA secure, then the EC-point Merkle proof-of-retention scheme is (t', ϵ) -sound with $\epsilon = \epsilon_1 + \epsilon_2 + 2^{-\lambda}$ and $t' \approx t$.*

Proof. We proceed by reduction. Suppose adversary A wins the EC-point PoR game with probability $\epsilon > \epsilon_1 + \epsilon_2 + 2^{-\lambda}$. A has deleted block $c_{j,i}$ and must produce $(\sigma', P', s'_1, \dots, s'_d)$ such that: (1) σ' is a valid ECDSA signature on m under public key P' , and (2) $H(P')$ hashes through the sibling path to $r_{M,j}$.

Case 1: $P' = P_{j,i}$. The adversary must produce a valid signature under $P_{j,i} = c_{j,i} \times G$ without knowing the private key $c_{j,i}$ (having deleted it). This constitutes an existential forgery under ECDSA, succeeding with probability at most ϵ_2 .

Case 2: $P' \neq P_{j,i}$ but $H(P') = H(P_{j,i})$. The adversary has found a collision in H , succeeding with probability at most ε_1 .

Case 3: $H(P') \neq H(P_{j,i})$ but the Merkle proof still verifies. By the same argument as Theorem 2, Case 1, this implies a collision at some internal Merkle node, with probability at most ε_1 .

By the union bound, $\Pr[A \text{ wins}] \leq \varepsilon_1 + \varepsilon_2 + 2^{-\lambda}$, where the $2^{-\lambda}$ term accounts for the probability that A guesses $c_{j,i}$ exactly (a 256-bit scalar). For SHA-256 and secp256k1, $\varepsilon_1 \leq 2^{-128}$, $\varepsilon_2 \leq 2^{-128}$ (under the generic group model), and $2^{-\lambda} = 2^{-128}$, giving $\varepsilon \leq 2^{-126}$. $\square$

Theorem 9 (Zero-Knowledge Property of EC-Point Proofs). *Under the ECDLP assumption, the EC-point proof transcript $(\sigma, P_{j,i}, s_1, \dots, s_d)$ reveals no information about the ciphertext block $c_{j,i}$ beyond what is implied by $P_{j,i} = c_{j,i} \times G$.*

Proof. The on-chain transcript consists of: (a) the public key $P_{j,i}$, (b) an ECDSA signature σ on a public message m , and (c) a sequence of hash values $(s_1, \dots, s_d)$. Recovering $c_{j,i}$ from $P_{j,i}$ requires solving the ECDLP. The signature σ reveals no additional information about the private key beyond its existence (standard ECDSA zero-knowledge property). The sibling hashes are precommitted in the Merkle tree and contain no information about $c_{j,i}$ beyond the tree structure. Therefore, the view of any polynomial-time on-chain observer is simulatable given only $P_{j,i}$ and the public Merkle tree structure, without knowledge of $c_{j,i}$. $\square$

Theorem 10 (Non-Transferability). *Under the ECDLP assumption and the encrypted multi-copy scheme, a valid EC-point proof for provider SP_j cannot be used by provider SP_k ($k \neq j$) to satisfy a challenge against T_k .*

Proof. Provider SP_k 's tree T_k has leaves $\text{leaf}_{k,i} = H(c_{k,i} \times G) = H(P_{k,i})$. Since $c_{j,i} \neq c_{k,i}$ (by per-copy encryption with independent keys), $P_{j,i} \neq P_{k,i}$, and $H(P_{j,i}) \neq H(P_{k,i})$ with overwhelming probability. The signature σ valid under $P_{j,i}$ is invalid under $P_{k,i}$ (ECDSA binding property). Therefore SP_k cannot reuse SP_j 's proof. $\square$

Interaction between Merkle hashing and EC key derivation. A potential concern is that the composition of hashing (H for Merkle leaves) and elliptic curve scalar multiplication (for key derivation) could introduce subtle interactions. We note that the two operations are applied sequentially, not compositionally: the Merkle leaf is $H(P)$, where $P = c \times G$ is computed first and then hashed. The hash function H treats P (an encoded elliptic curve point, 33 bytes compressed) as an opaque bit string. No algebraic relationship between the EC group operation and the hash function is exploited or required. The security reduction therefore decomposes

cleanly: ECDSA unforgeability protects the signature step, and collision resistance of H protects the Merkle verification step. There is no known generic attack that exploits the composition of these primitives, and the construction follows the standard hash-then-sign paradigm used in deployed cryptographic protocols (e.g., TLS certificate chains, Bitcoin transaction signing).

6. Formal Analysis

6.1. Storage Cost Model

Let D denote total data volume, n the number of blocks (each size $s = D/n$), $N = |V_s|$ storage nodes, and r the replication factor. Under full replication: $S_{\text{full}} = N \times D$. Under DHT partitioning: $S_{\text{DHT}} = r \times D$, with per-node storage $r \times D/N$. The reduction factor is N/r . Table 5 provides a numerical comparison for representative parameters.

Table 5. Storage cost comparison ($D = 1 \text{ TB}$, $N = 100 \text{ nodes}$)

Parameter	Full Replication	DHT ($r = 2$)
Total data	1 TB	1 TB
Aggregate storage	100 TB	2 TB
Per-node storage	1 TB	20 GB
Reduction factor	—	50×
Network ingress	100 TB	2 TB

The storage efficiency gain is linear in N/r . For 100 nodes with $r = 2$, aggregate storage is reduced by 50×. For 1,000 nodes with $r = 3$, the reduction is approximately 333×. These gains are independent of the data volume, making the architecture equally applicable to small enterprise datasets and multi-petabyte archives.

6.2. Proof Verification Complexity

The Merkle proof for a leaf in a tree with n leaves has depth $d = \lceil \log_2 n \rceil$. Proof size is $d \times 32$ bytes (SHA-256). Table 6 shows proof sizes across representative data volumes.

Table 6. Merkle proof size by data item size (1 KB blocks, SHA-256)

Data size	Blocks (n)	Depth (d)	Proof size
1 MB	1,024	10	320 B
1 GB	1,048,576	20	640 B
1 TB	$\sim 10^9$	30	960 B
1 PB	$\sim 10^{12}$	40	1,280 B

Proof size grows logarithmically with data volume. A 1 TB dataset requires only 960 bytes of proof—negligible in a standard blockchain transaction. On-chain verification cost is also logarithmic: approximately $d \times 4 = 120$ opcodes for a depth-30 tree. This is within the script execution limits of standard UTXO-based blockchains, requiring no protocol modifications.

6.3. DHT Availability Under Node Churn

With replication factor r and independent node failures, a data block is unavailable only if all r replicas fail simultaneously. For hourly failure probability $p = 1/(24 \times 365) \approx 1.14 \times 10^{-4}$: $P_{\text{fail}} = p^r$. Table 7 shows availability levels across replication factors.

Table 7. DHT data block availability ($p = 1.14 \times 10^{-4}/\text{hour}$)

r	P_{fail}	Availability	Annual downtime
1	1.14×10^{-4}	99.989%	~57 min
2	1.30×10^{-8}	99.9999987%	~0.004 s
3	1.49×10^{-12}	~100%	Negligible

At $r = 2$, expected annual downtime per block is ~0.004 seconds—exceeding enterprise “five nines” (99.999%) guarantees with aggregate storage of only 2D. This demonstrates that DHT-based partitioning with modest replication achieves availability comparable to enterprise storage systems without the $O(N \times D)$ cost of full replication.

6.4. DHT Routing Latency

Under Pastry, expected hops = $\lceil \log_{16} N \rceil$. Total lookup latency $\leq \lceil \log_{16} N \rceil \times \text{RTT}$. Table 8 shows latency across network sizes.

Table 8. DHT routing latency ($\text{RTT} = 10 \text{ ms}$)

Nodes (N)	Hops	Latency
100	2	20 ms
1,000	3	30 ms
10,000	4	40 ms
100,000	5	50 ms

For 1,000 nodes with 10 ms RTT, lookup completes in ~30 ms—negligible compared to data transfer time for multi-kilobyte blocks. The logarithmic scaling ensures that routing overhead remains bounded even as the network grows to tens of thousands of nodes.

6.5. Collusion Resistance Formal Model

Theorem 1 (Collusion Resistance). *Under the encrypted multi-copy scheme with independently generated keys, no coalition of $m - 1$ storage providers can produce a valid*

retention proof for the m -th provider without possessing the m -th provider's ciphertext, assuming H is collision-resistant and the encryption scheme is IND-CPA secure.

Proof sketch. A valid proof for SP_m requires pre-image b' such that $H(b') = \text{leaf}_{m,i} = H(c_{m,i})$. Since $c_{m,i} = b_i \oplus K_{m,i}$ and $K_{m,i}$ is unknown to all storage providers, producing $b' = c_{m,i}$ requires knowledge of $K_{m,i}$. A coalition holding $C_1, \dots, C_{m-1}$ cannot derive C_m by IND-CPA security. The coalition must find a hash collision (contradicting collision resistance) or obtain C_m directly—the behaviour the system enforces. $\square$

7. Advanced Data Distribution Strategies

7.1. RAID-Inspired Data Striping

The architecture supports RAID-inspired configurations for parallel retrieval and fault tolerance. In a RAID 0+1 configuration, data Δ is striped into halves ($\Delta_{\text{odd}}, \Delta_{\text{even}}$), each mirrored to produce four segments, each encrypted separately with distinct keys. Each ciphertext is mapped into an independent Merkle tree and assigned to a different storage provider. This enables parallel retrieval (doubling bandwidth) and single-failure tolerance. More generally, any RAID configuration (including RAID 5 with distributed parity) can be applied before encryption and distribution, allowing the data provider to select the trade-off between redundancy level, retrieval performance, and storage cost that best suits their requirements.

7.2. Race-Based Proofs

An alternative to provider-specific challenges creates a race condition: the locking script can be unlocked by any provider who possesses the challenged data and produces a valid proof. The first responder claims the output value, creating competitive pressure for high-availability infrastructure and rapid response times. Race-based proofs are useful when multiple providers store unencrypted data for retrieval redundancy, and during data retrieval where providers compete to deliver data fastest. A hybrid approach allocates a base fee to all responders within a deadline and a bonus to the first, balancing reliability incentives with competitive pressure.

7.3. Data Retrieval Mechanisms

When a client requests data retrieval, the service provider locates the responsible storage node(s) via DHT lookup and initiates transfer. Data can be sent off-chain (preserving privacy and avoiding large on-chain transactions) or on-chain (providing an immutable transfer record). For encrypted multi-copy storage with RAID striping, parallel retrieval from multiple providers is supported: the client simultaneously downloads different stripes from different providers, with the service provider reassembling and decrypting the complete data item. Previously

submitted Merkle proofs can be reused to shorten new proof lengths during retrieval verification, reducing the overhead of combining retrieval with integrity verification.

8. Security Model and Proofs

8.1. Definitions

We formalise the proof-of-retention protocol as a game between a challenger C (the service oracle) and an adversary A (a potentially dishonest storage provider), following the provable data possession framework of [17] and adapted for the on-chain verification setting.

Definition 1 (Proof-of-Retention Game). Let λ be a security parameter. The PoR game proceeds in four phases:

Setup(λ). The challenger C selects a collision-resistant hash function $H: \{0,1\}^* \rightarrow \{0,1\}^{256}$ (instantiated as SHA-256). C divides data Δ into n blocks $b_1, \dots, b_n$, constructs Merkle tree T with root r_M , and publishes r_M to the blockchain. C sends the full data Δ to A .

Challenge(i). C selects $i \leftarrow [1, n]$ uniformly at random and constructs challenge transaction T_{chal} with locking script embedding $(r_M, i, \text{verification}_{\text{logic}})$.

Response. A produces response transaction T_{resp} with unlocking script containing purported proof $\pi = (b', s'_1, \dots, s'_d)$ where b' is the purported block and s'_j are the purported sibling hashes.

Verify. The blockchain script engine executes the combined script. The proof is accepted if and only if iterative hashing of π from the leaf to the root yields r_M .

Definition 2 (Possession Soundness). The PoR scheme is (ϵ, q) -sound if: for any PPT adversary A that has deleted at least one block b_j from its local storage, the probability that A wins the PoR game (i.e. produces a valid proof when challenged on block j) is at most ϵ , over the random choice of challenge index, after q challenge-response rounds.

Definition 3 (Outsourcing Resistance). Let $B(D, \tau)$ denote the bandwidth required to retrieve block b_i from any external source within the challenge deadline τ . The scheme is (τ, B) -outsourcing-resistant if any adversary A that does not locally store b_i must transfer at least $|b_i|$ bytes within time τ to produce a valid proof, and the protocol sets τ such that $B(D, \tau)$ exceeds A 's available bandwidth with overwhelming probability.

8.2. Theorem 2: Possession Soundness Under Collision Resistance

Theorem 2. If H is (t, ϵ_I) -collision-resistant, then the Merkle proof-of-retention scheme is (t', ϵ) -sound with $\epsilon = \epsilon_I + 2^{-\lambda}$ and $t' \approx t$.

Proof. We proceed by reduction. Assume adversary A wins the PoR game with probability $\epsilon > \epsilon_1 + 2^{-\lambda}$. We construct a collision-finder B that uses A to find collisions in H.

Construction of B. B receives a hash function H and must output a collision (x, x') with $H(x) = H(x')$ and $x \neq x'$. B simulates the PoR game for A as follows: B runs Setup honestly, constructing a valid Merkle tree T over blocks $b_1, \dots, b_n$. B gives Δ to A, then waits. At challenge time, B selects random index i and issues Challenge(i). A, having deleted block b_i , responds with purported proof $\pi = (b', s'_1, \dots, s'_d)$.

Extraction of collision. If the script engine accepts π (the combined script evaluates to TRUE), then the iterative hashing of $(b', s'_1, \dots, s'_d)$ from the leaf to the root produces r_M . B knows the correct proof path $(b_i, s_1, \dots, s_d)$ which also hashes to r_M . Since A deleted b_i , A does not possess b_i and must use $b' \neq b_i$ (with probability $1 - 2^{-\lambda}$ over the randomness of the challenge). There are two cases:

Case 1: $H(b') \neq H(b_i)$ but the proof still verifies. Then there exists some level j in the Merkle tree where the computed intermediate hash from A's proof diverges from the correct intermediate hash, yet both paths ultimately produce r_M . This means two distinct inputs to H at level j produced the same output—B outputs this pair as a collision.

Case 2: $H(b') = H(b_i)$ but $b' \neq b_i$. B outputs (b', b_i) as a collision in H directly.

In both cases, whenever A produces a valid proof without possessing b_i , B obtains a collision in H. Therefore $\Pr[A \text{ wins}] \leq \epsilon_1 + 2^{-\lambda}$, where ϵ_1 is the collision-finding advantage and $2^{-\lambda}$ accounts for the probability that A guesses b_i exactly. For SHA-256 with $\lambda = 128$, this gives $\epsilon \leq 2^{-128} + 2^{-128} = 2^{-127}$. $\square$

8.3. Theorem 3: Outsourcing Resistance Under Bandwidth Constraints

Theorem 3. *If the challenge deadline τ satisfies $\tau < |b| / BW_{\max} + RTT_{\min}$, where $BW_{\max}$ is the adversary's maximum download bandwidth to any external data source and $RTT_{\min}$ is the minimum round-trip time to any node holding the data, then no adversary without local possession of b_i can respond to a challenge on block i within deadline τ .*

Proof. A storage provider who has deleted block b_i must retrieve it from some source to construct a valid proof. Under the encrypted multi-copy scheme, provider SP_j holds $C_j = \text{Enc}(K_j, \Delta)$, and no other provider SP_k ($k \neq j$) holds C_j (each holds a distinctly encrypted copy). Therefore SP_j cannot retrieve $c_{j,i}$ from any other storage provider. SP_j could attempt to retrieve $c_{j,i}$ from the service provider (which holds all keys), but this requires: (a) identifying the

challenged index before the deadline, (b) requesting and receiving $|b_i|$ bytes within time τ , and (c) constructing the response transaction. If τ is set below the physical transfer time $|b_i|/BW_{\max} + RTT_{\min}$, this is impossible regardless of A's computational resources.

For concrete parameters: with 1 KB blocks, 1 Gbps adversary bandwidth, and 20 ms minimum RTT, the transfer lower bound is $0.008 \text{ ms} + 20 \text{ ms} \approx 20 \text{ ms}$. Setting $\tau = 10$ seconds provides a $500\times$ margin for honest providers (who perform only a local disk read of $\sim 0.1 \text{ ms}$) while remaining tight enough that an adversary would need sustained $>100 \text{ Mbps}$ to the specific data source holding their encrypted copy. When τ is combined with the encrypted multi-copy scheme (where no external source holds the provider's specific ciphertext), the outsourcing attack becomes infeasible even with unlimited bandwidth. $\square$

8.4. Theorem 4: Selective Retention Detection

Theorem 4 (Challenge Coverage). *If challenge indices are selected uniformly and independently from $[1, n]$ over k epochs, then the probability that a specific block b_j has been challenged at least once is:*

$$P_{\text{detect}}(j, k) = 1 - (1 - 1/n)^k \geq 1 - e^{-k/n}$$

Corollary. For $n = 10^6$ blocks and a target detection probability of $1 - \delta$:

Table 9. Challenge epochs required for detection probability $P \geq 1 - \delta$ ($n = 10^6$ blocks)

Target P_{detect}	δ	Required k	At 1 chal/day	At 10 chal/day
0.99	0.01	4.6×10^6	$\sim 12,600 \text{ yrs}$	$\sim 1,260 \text{ yrs}$
0.99	0.01 (any 1 of n)	$4.6n$	$\sim 12.6 \text{ yrs}$	$\sim 1.26 \text{ yrs}$
0.999	0.001 (any 1 of n)	$6.9n$	$\sim 18.9 \text{ yrs}$	$\sim 1.89 \text{ yrs}$
$0.63 (1 - e^{-1})$	0.37 (per block)	n	$\sim 2,740 \text{ yrs}$	$\sim 274 \text{ yrs}$

The crucial distinction is between challenging a *specific* block (astronomically slow) and detecting *any* deletion of at least one block from a set (feasible). An adversary who deletes a fraction α of blocks fails a random challenge with probability α per epoch. After k epochs, the probability of detecting deletion is $1 - (1 - \alpha)^k$. For $\alpha = 0.01$ (1% of blocks deleted) and $k = 460$, detection probability exceeds 0.99. At one challenge per day, a storage provider deleting 1% of its assigned data is detected within approximately 460 days (15 months) with 99% confidence. At 10 challenges per day, detection occurs within 46 days. This detection probability applies per provider: with 100 providers each challenged independently, any individual deletion is detected with the same per-provider bounds.

8.5. Formalised Oracle Trust Model

The security analysis in Sections 8.2–8.4 assumes an honest oracle (service provider). We now relax this assumption by defining three oracle trust levels and analysing security under each.

Definition 5 (Oracle Trust Levels).

Semi-honest oracle. The oracle follows the protocol specification correctly—constructing valid Merkle trees, issuing random challenges, and enforcing deadlines—but may attempt to learn information about stored data beyond what is revealed by the protocol. The semi-honest oracle does not deviate from prescribed behaviour, forge challenges, or collude with storage providers.

Malicious oracle. The oracle may deviate arbitrarily from the protocol: issuing non-random challenge indices, constructing invalid Merkle trees, failing to issue challenges to favoured providers, or selectively withholding challenge transactions. The malicious oracle acts alone without coordination with storage providers.

Colluding oracle. The oracle coordinates with a subset $S \subseteq \{SP_1, \dots, SP_m\}$ of storage providers. The coalition shares private information including encryption keys K_j for $j \in S$, challenge schedules, and proof data. This is the strongest adversary model and subsumes the previous two.

Theorem 5 (Security Under Semi-Honest Oracle). *Under a semi-honest oracle, Theorems 2–4 hold as stated. Additionally, the oracle learns no information about stored data beyond: (a) the block sizes, (b) the Merkle root, and (c) the timing of provider responses.*

Proof. Under the ciphertext-only proof variant (Section 4.6), the unlocking script reveals $c_{j,i} = b_i \oplus K_{j,i}$. Since the oracle constructed $K_{j,i}$, it can recover b_i from the on-chain ciphertext. This is inherent: the oracle prepared the data and holds all keys. Under the EC-point proof variant, the unlocking script reveals only an ECDSA signature and the public key $P_{j,i} = c_{j,i} \times G$. Even though the oracle knows $K_{j,i}$, it already possesses the plaintext by construction. The relevant privacy property is against third-party observers, not the oracle. Challenge randomness is guaranteed by derivation from the block hash (Section 4.4), which the semi-honest oracle cannot influence. The semi-honest oracle therefore provides no additional attack surface beyond its inherent key-holder status. $\square$

Theorem 6 (Security Under Malicious Oracle). *Under a malicious oracle, possession soundness (Theorem 2) and selective-retention detection (Theorem 4) remain sound if*

challenge index derivation is publicly verifiable. Outsourcing resistance (Theorem 3) holds provided the oracle does not supply ciphertext blocks to providers.

Proof. We address each attack vector.

Non-random challenge selection. If the oracle selects challenge indices adversarially (e.g., repeatedly challenging blocks it knows the provider retains), detection coverage degrades. Defence: challenge indices are derived deterministically as $i = H(\text{block}_h \parallel \text{epoch} \parallel \text{provider}_{id}) \bmod n$, where block_h is the hash of the most recent blockchain block at challenge issuance. Any third-party observer can recompute the expected index and compare it against the oracle's published challenge transaction. A malicious oracle that deviates from the deterministic derivation is publicly detectable. Under this constraint, Theorem 4 holds because challenge indices inherit the pseudorandomness of the block hash.

Forged Merkle trees. A malicious oracle could commit a Merkle root r'_M over a dataset $\Delta' \neq \Delta$. This is detectable only if the data provider or a third party independently verifies the root at commitment time. We assume the data provider verifies r_M before accepting the storage contract. Under this assumption, the committed root is correct and immutable thereafter.

Challenge suppression. A malicious oracle may fail to issue challenges to favoured providers. Since challenge transactions are on-chain and publicly observable, any auditor can detect missing challenges by comparing the expected challenge schedule (deterministic from block hashes) against the transaction record. We define a challenge-completeness monitor as any node that verifies: for each epoch e and provider SP_j , a challenge transaction exists with index $i = H(\text{block}_{he} \parallel e \parallel j) \bmod n$. Deviation is publicly provable. $\square$

Theorem 7 (Security Under Colluding Oracle–Provider Coalition). *Under a colluding coalition of the oracle and a subset S of storage providers, possession soundness is maintained for non-colluding providers SP_j ($j \notin S$). Outsourcing resistance for colluding providers SP_j ($j \in S$) is defeated. Detection of the coalition is possible through public auditability of challenge–response timing.*

Proof. A colluding oracle can supply $c_{j,i}$ directly to $SP_j \in S$, enabling SP_j to produce valid proofs without local storage. This defeats outsourcing resistance for colluding providers. However, the attack is constrained by three factors.

(i) *Timing observability.* For the oracle to supply $c_{j,i}$ to SP_j after challenge issuance, it must transmit $|b_i|$ bytes within deadline τ . The response transaction's confirmation time is publicly

recorded. Under deterministic challenge derivation, the challenge index becomes known at block_h publication. A colluding provider's response time distribution will exhibit a lower bound equal to the oracle-provider communication latency, distinguishable from an honest provider's local-read response time. Let T_{honest} denote the honest response time (dominated by local disk I/O, typically <1 ms) and $T_{\text{collusion}}$ the colluding response time (dominated by network round-trip, typically >3 ms even on a LAN). A statistical test comparing response time distributions across epochs can distinguish the two cases with power increasing in the number of observed epochs (see Section 9.7.3 for simulation validation).

(ii) *Independence across providers.* The coalition gains no advantage against non-colluding providers. Proofs from SP_k ($k \notin S$) remain sound under Theorem 2, because the oracle cannot forge proofs for trees it committed honestly.

(iii) *Oracle decentralisation.* The oracle role can be distributed among m independent entities using t -of- m threshold key management for encryption keys. Under this extension, collusion requires corrupting at least t independent oracle shards plus the target storage provider. For $t = 3$ and $m = 5$, the coalition must compromise at least 4 independent entities. $\square$

8.6. Collusion Case Summary

Table 10 summarises the security guarantees under each oracle trust model.

Table 10. *Security guarantees under three oracle trust models*

Oracle Model	Soundness	Outsourcing Resistance	Detection	Required Mitigation
Semi-honest	Full (Thm 2)	Full (Thm 3)	Full (Thm 4)	None
Malicious (solo)	Full	Full	Conditional*	Deterministic index derivation; root verification
Colluding (oracle + S)	Full ($j \notin S$)	Defeated ($j \in S$)	Timing analysis	Threshold oracle (t -of- m); response timing audit

Conditional on publicly verifiable challenge derivation.

The principal design implication is that the oracle-provider collusion case, while not cryptographically preventable under a single-oracle architecture, is (a) publicly detectable through timing analysis and challenge-completeness monitoring, and (b) structurally mitigable through threshold oracle decentralisation. Future implementations should treat threshold oracle deployment as a priority for high-assurance applications.

8.7. Threat Model Summary

Table 11 summarises the assumptions and adversary capabilities across all protocol layers.

Table 11. *Threat model summary: honest, adversarial, and assumed components*

Component	Assumption	Adversary capability	Defence mechanism
Blockchain consensus	Honest majority of mining power	Cannot rewrite confirmed blocks	Standard Nakamoto consensus [38]
Hash function (SHA-256)	Collision resistance	Cannot find collisions in PPT	Theorem 2 reduction
DHT routing	Honest majority of storage nodes	Eclipse, Sybil, churn manipulation	VRF identifiers; reputation; deposits (§3.5–3.6)
Service provider (oracle)	Semi-honest (default); may be malicious or colluding	Bias challenges; forge trees; supply proofs to coalition	Deterministic derivation; root verification; timing audit; threshold oracle (§8.5–8.6)
Storage providers	Rational and potentially adversarial	Delete data; outsource storage; collude with oracle	Merkle proofs (Thm 2); bandwidth bound (Thm 3); detection (Thm 4); timing analysis (Thm 7)
Network	Asynchronous with bounded delay	Delay or reorder messages (but not forge)	Challenge deadlines; blockchain finality
Data provider	Trusted at setup	N/A (defines the stored data)	Verifies Merkle root at commitment

9. Systems Evaluation

We present a quantitative evaluation of the protocol across three dimensions: on-chain verification cost on the BSV blockchain, economic sustainability under realistic storage parameters, and DHT resilience under node churn. All measurements use analytically derived closed-form bounds validated against the BSV scripting specification; the purpose is to demonstrate implementability and cost viability rather than to report empirical throughput benchmarks, which require a deployed prototype.

9.1. On-Chain Verification Cost

Table 12 presents the complete cost model for a single challenge-response cycle on BSV, broken down by transaction component. The model assumes standard BSV transaction structure: version (4 bytes), input count (1), output count (1), locktime (4), and per-input/output overhead.

Table 12. *Complete BSV transaction cost breakdown for one challenge-response cycle*

Component	Bytes	Notes
Challenge Tx (T_{chal})		

Version + locktime	8	Standard
Input (prev UTXO)	~148	txid(32) + vout(4) + scriptSig(~105) + seq(4)
Output (locking script)	6d + 35 + 9	d×(6 opcodes) + root(32) + OP_EQUAL + overhead
Output (change)	~34	Standard P2PKH
Total T_{chal}	199 + 6d	d=20: 319 B; d=30: 379 B
Response Tx (T _{resp})		
Version + locktime	8	Standard
Input (spending T _{chal})	33d + 32 + 41	d sibling hashes(33d) + block hash(32) + overhead
Output (payment)	~34	Standard P2PKH
Total T_{resp}	83 + 33d	d=20: 743 B; d=30: 1,073 B
Cycle total	282 + 39d	d=20: 1,062 B; d=30: 1,452 B
BSV fee (1 sat/B)	282 + 39d sat	d=20: ≈\$0.00053; d=30: ≈\$0.00073

9.2. Economic Sustainability Model

We model the economics for three deployment scenarios representing small, medium, and large storage networks. The parameters are: number of storage providers N , total stored data D , replication factor $r = 2$, challenge frequency f challenges per provider per day, and Merkle tree depth $d = \lceil \log_2(D/\text{block_size}) \rceil$ with 1 KB block size. BSV fee rate is assumed at 1 satoshi/byte with BSV/USD = \$50.

Table 13. Annual on-chain verification cost for three deployment scenarios

Parameter	Small	Medium	Large
Storage providers N	10	100	1,000
Total data D	100 GB	10 TB	1 PB
Blocks n	10^8	10^{10}	10^{12}
Tree depth d	27	34	40
Per-node storage	20 GB	200 GB	2 TB
Challenges/day/provider	1	1	1
Cycles/day	10	100	1,000
Bytes/cycle	1,335 B	1,608 B	1,842 B
Daily on-chain bytes	13.0 KB	157.0 KB	1.80 MB
Annual on-chain bytes	4.87 MB	58.7 MB	672 MB
Annual cost (sat)	4.87M	58.7M	672M
Annual cost (USD)	\$2.44	\$29.35	\$336
Cost per TB per year	\$24.36	\$2.94	\$0.33

Verification cost per TB per year decreases sharply with scale: from \$24.36 at 100 GB to \$0.33 at 1 PB. For comparison, Amazon S3 storage alone (excluding retrieval fees) costs approximately \$276 per TB per year (standard tier, us-east-1). The entire verification overhead for a petabyte-scale deployment (\$336/year) represents approximately 0.12% of the equivalent S3 storage cost (\$276,000/year for 1 PB), confirming that on-chain verification is economically negligible relative to the storage costs it secures.

9.3. Deposit and Payment Schedule

The verify-then-pay mechanism operates through a structured deposit and payment schedule that prevents single-epoch-and-disappear behaviour:

Initial deposit. Upon joining a storage contract, provider SP_j posts a deposit D_{dep} to a time-locked UTXO that can only be spent after the contract duration T_{contract} . The deposit is set to $D_{\text{dep}} = \alpha \times T_{\text{contract}} \times f_{\text{epoch}}$, where α is the per-epoch payment and f_{epoch} is the challenge frequency, ensuring the deposit covers multiple missed challenges.

Epoch payments. Each successful challenge-response unlocks a payment UTXO worth α satoshis. The service provider pre-funds the challenge outputs for the next payment period (e.g., 30 days), creating a batch of 30 time-locked challenge UTXOs in a single funding transaction. The storage provider claims each payment by producing a valid Merkle proof within the deadline.

Slashing on missed challenges. If SP_j fails to respond to a challenge within deadline τ , the challenge UTXO reverts to the service provider (via a timeout branch in the locking script: `OP_CHECKSEQUENCEVERIFY`). After q consecutive missed challenges, a fraction of D_{dep} is forfeited (burned to an unspendable output or returned to the data provider). This makes sustained non-responsiveness economically self-destructive.

Contract renewal. At the end of T_{contract} , if all challenges have been answered, D_{dep} is returned to SP_j and a new contract period begins. This ensures that the deposit acts as a bond for ongoing retention, not merely initial possession.

9.4. DHT Churn Resilience Analysis

We analyse the DHT overlay's resilience to node churn using a Poisson failure model. Each storage node has an independent hourly failure probability $p = 1/(24 \times 365) \approx 1.14 \times 10^{-4}$ (corresponding to mean time between failures of approximately one year). When a node fails, its DHT keyspace responsibility is reassigned to leaf-set neighbours, and data blocks are re-replicated from surviving replicas.

Table 14. DHT resilience under node churn ($N = 1,000$ nodes, $D = 10$ TB, 1 KB blocks)

Metric	$r = 1$	$r = 2$	$r = 3$
P(block loss per hour)	1.14×10^{-4}	1.30×10^{-8}	1.49×10^{-12}
Expected blocks lost/year	$\sim 1.07 \times 10^{10}$	$\sim 580,000^*$	~ 0
Data loss per year	~ 10 TB (total)	~ 566 MB*	~ 0
Hourly block availability ($1 - P_{\text{loss}}$)	99.989%	99.9999987%	$\sim 100\%$
Re-replication bandwidth/day	N/A	~ 28 GB	~ 28 GB
Recovery time per event (100 Mbps)	N/A	~ 800 s	~ 800 s

Simulation-validated figures (Section 9.7.2). At $r = 1$, every node failure permanently destroys its partition (no replica exists), and with MTBF = 1 year across 1,000 nodes, effectively all data is lost within one year. This confirms that replication ($r \geq 2$) is essential. The analytical independence-based prediction underestimates block loss at $r = 2$ because it does not account for correlated failures of adjacent nodes within the re-replication window. The simulation captures these correlations, yielding higher but more realistic loss estimates. At $r = 3$, no permanent block loss was observed across 1,000 simulated years.

At $r = 2$, re-replication bandwidth is approximately 28 GB per day, reflecting the per-node partition size of $D/N = 10$ GB transferred per failure event with approximately 2.74 failures per day across the 1,000-node network. Each re-replication completes in approximately 800 seconds (~ 13 minutes) at 100 Mbps. For time-critical applications, provisioning dedicated re-replication bandwidth or using higher link speeds (1 Gbps reduces transfer to ~ 80 seconds per event) is advisable. At $r = 3$, expected data loss is effectively zero—no simulation trial recorded permanent block loss across 1,000 simulated years—confirming that $r \geq 3$ provides durability exceeding enterprise storage guarantees with aggregate storage of only $3D$. These results are consistent with the analytical models of [2] for Google’s distributed storage, where similar replication factors achieved comparable durability in production deployments.

9.5. Multicast Deployment Constraints

The original architecture specified IP multicast for S-net overlay communication. We acknowledge that end-to-end IP multicast is not universally available on the public internet, limiting this to deployments within managed network environments (e.g., data-centre LANs, enterprise WANs, or overlay networks using application-layer multicast). For public internet deployments, we specify a fallback: application-layer multicast via the DHT structure itself, where each node forwards received data to its leaf-set neighbours. Under Pastry, this achieves $O(\log N)$ dissemination latency with per-node fan-out bounded by the leaf-set size L . For $L =$

16 and $N = 1,000$, full dissemination requires at most 3 relay hops with each node forwarding to at most 16 peers. This is strictly less efficient than native IP multicast ($O(1)$ fan-out) but avoids any dependence on network-layer multicast support. UDP transport remains appropriate for the overlay regardless of the multicast mechanism, as the application-layer reliability protocol (acknowledgement tracking, selective retransmission) operates above the transport layer and is not dependent on multicast availability.

9.6. Fee Sensitivity Analysis

The cost projections in Sections 9.1–9.2 assume a BSV fee rate of 1 satoshi/byte and a BSV/USD exchange rate of \$50. Both parameters are subject to market fluctuation. We perform a sensitivity analysis to identify the conditions under which the economic model remains viable and the thresholds at which alternative mitigation strategies become necessary.

9.6.1. Fee rate sensitivity. We define the annual verification cost for the medium deployment scenario ($N = 100$ providers, $D = 10$ TB, $d = 34$) as $C_{\text{annual}} = N \times 365 \times (282 + 39d) \times f \times R$, where f is the fee rate in satoshis per byte and R is the BSV/USD exchange rate in dollars per satoshi. Table 15 presents annual verification cost under varying fee rates.

Table 15. Annual verification cost sensitivity to fee rate (medium scenario: $N = 100$, $D = 10$ TB, BSV/USD = \$50)

Fee Rate (sat/B)	Cost/Cycle	Annual Cost (USD)	Cost/TB/Year	% of S3 Equiv.
1 (baseline)	\$0.00080	\$29.35	\$2.94	1.06%
5	\$0.0040	\$146.75	\$14.68	5.32%
10	\$0.0080	\$293.50	\$29.35	10.6%
50	\$0.040	\$1,468	\$146.75	53.2%
100	\$0.080	\$2,935	\$293.50	106%

S3 equivalent: \$276/TB/year (standard tier, us-east-1).

The economic model remains compelling ($\leq 10\%$ of equivalent cloud costs) up to approximately $10\times$ the baseline fee rate. The protocol reaches economic parity with S3 at approximately 94 sat/byte—nearly two orders of magnitude above current BSV fee rates, which typically operate below 1 sat/byte. We note that the S3 benchmark represents hyperscaler pricing; decentralised storage markets may diverge from hyperscaler cost structures in the longer term. Furthermore, challenge batching (Section 9.6.3) extends this margin substantially.

9.6.2. Exchange rate sensitivity. Since fees are denominated in satoshis and costs realised in fiat, appreciation of BSV increases dollar-denominated verification costs proportionally. Table 16 presents sensitivity to BSV/USD exchange rate.

Table 16. *Annual cost and break-even fee rate under varying BSV/USD exchange rates (medium scenario, 1 sat/byte base fee)*

BSV/USD	Annual Cost	Cost/TB/Year	% of S3	Break-Even Fee
\$10	\$5.87	\$0.59	0.21%	470 sat/B
\$50 (baseline)	\$29.35	\$2.94	1.06%	94 sat/B
\$200	\$117.40	\$11.74	4.25%	24 sat/B
\$1,000	\$587	\$58.70	21.3%	5 sat/B
\$5,000	\$2,935	\$293.50	106%	<1 sat/B

Break-even fee is the fee rate at which verification cost equals S3 equivalent storage cost.

The break-even fee column shows the fee rate at which verification cost matches S3 pricing at each exchange rate. Even at BSV/USD = \$1,000 (a 20× appreciation), the break-even fee is 5 sat/byte. Above this exchange rate, the protocol benefits from challenge batching (Section 9.6.3) or reduced challenge frequency to maintain economic viability. The model becomes stressed only under the joint condition of high exchange rates and high fee rates simultaneously.

9.6.3. Batching as mitigation. Under sustained high-fee conditions, challenge batching provides a structural mitigation. Instead of issuing N individual challenge transactions per day, the oracle batches B challenges into a single transaction with B outputs (one per challenged provider). The transaction overhead (version, locktime, input) is amortised across B challenges, reducing per-challenge cost. For $d = 34$ and $B = 100$, overhead amortisation reduces per-challenge cost by approximately 17%. Batching does not alter the security properties of the protocol: each output embeds an independent Merkle root and challenge index, and each response is independently verified.

Additionally, challenge frequency can be dynamically adjusted as a function of observed fee rates. Reducing challenge frequency from 1/day to 1/week decreases annual cost by 7×, at the expense of slower detection (Theorem 4). For archival storage where detection within one year is acceptable, weekly challenges at even 50× baseline fee rates yield annual costs below \$210 for the medium scenario—still economically viable.

9.6.4. Upper bound fee tolerance. We define the critical fee threshold f^* as the fee rate at which annual verification cost exceeds annual equivalent S3 storage cost: $f^* = (S3_{eq} \times D) / (N \times 365 \times (282 + 39d) \times R)$. For the medium scenario at BSV/USD = \$50: $f^* \approx 94$ sat/byte. This confirms that the economic model is robust to fee fluctuations of nearly two orders of magnitude above current conditions.

9.7. Simulation Validation

To complement the analytical results in Sections 6 and 9.1–9.5, we present results from three discrete-event simulations validated against the theoretical models: (i) selective-retention detection under varying deletion fractions; (ii) DHT churn and re-replication dynamics; and (iii) challenge-response timing distributions under honest and outsourcing-attack scenarios. All simulations are implemented in Python 3 using NumPy (random number generation, statistical aggregation) and SciPy (Kolmogorov–Smirnov tests). Each simulation uses a fixed random seed (seed = 42) for exact reproducibility. Source code is provided as Supplementary Material.

9.7.1. Selective-retention detection simulation. We simulate the challenge-response game (Definition 1) for $n = 10^6$ blocks with deletion fraction $\alpha \in \{0.001, 0.005, 0.01, 0.05, 0.10\}$. At each epoch, a block index is selected uniformly at random; if the selected block has been deleted, the challenge fails and the provider is detected. We record the number of epochs to first detection across 10,000 independent trials per deletion fraction.

Two implementations are executed in parallel: (i) a fast path using the geometric distribution (mathematically equivalent to uniform random challenge against Bernoulli deletion), generating 10,000 samples per configuration; and (ii) an explicit block-level simulation (marking specific blocks as deleted, drawing uniform random indices, checking membership) for 500–1,000 trials per configuration as cross-validation. Table 17 compares simulated results against the analytical prediction $k^* = \ln(1/\delta)/\alpha$ from Theorem 4.

Table 17. *Selective-retention detection: analytical vs. simulated (10,000 geometric trials + 500–1,000 explicit trials per α ; seed = 42)*

α (deleted)	Analytical k^* ($\delta=0.01$)	Sim. median	Sim. P95	Sim. P99	Explicit P99	Rel. error (P99)
0.001 (0.1%)	4,605	686	2,985	4,573	4,680	0.70%
0.005 (0.5%)	921	142	621	941	940	2.17%
0.01 (1%)	461	69	293	461	488	0.11%

α (deleted)	Analytical k^* ($\delta=0.01$)	Sim. median	Sim. P95	Sim. P99	Explicit P99	Rel. error (P99)
0.05 (5%)	92	14	60	93	88	0.97%
0.10 (10%)	46	7	29	43	46	6.63%

Analytical k is the epoch count for 99% detection probability. Relative error is $|\text{analytical} - \text{simulated P99}| / \text{analytical}$.

The simulated 99th-percentile detection times match the analytical predictions within 2.2% for deletion fractions $\alpha \in \{0.005, 0.01, 0.05\}$, and within 6.7% at $\alpha = 0.10$ where the small absolute value of k^* (46 epochs) amplifies discrete-distribution granularity. At $\alpha = 0.001$, the relative error is 0.7%. The explicit block-level simulation cross-validates the geometric approximation: explicit P99 values agree with geometric P99 values within 8% across all tested configurations. The median detection time is consistently faster than the 99th percentile by a factor of approximately $\ln(100)/\ln(2) \approx 6.6\times$, reflecting the heavy right tail of the geometric distribution. These results confirm that the analytical model (Theorem 4) provides accurate, slightly conservative bounds on detection time.

9.7.2. DHT churn and re-replication simulation. We simulate a Pastry DHT with $N = 1,000$ nodes, $D = 10$ TB (10,737,418,240 blocks of 1 KB), and replication factors $r \in \{1, 2, 3\}$. Node failures are generated as independent Poisson events with rate $\lambda = 1/(24 \times 365)$ per node per hour (MTBF ≈ 1 year). Upon failure detection (heartbeat interval $\Delta = 30$ seconds), surviving leaf-set neighbours initiate re-replication of the failed node's partition from the nearest surviving replica. The re-replication link speed is 100 Mbps. We simulate 8,760 hours (1 year) across 1,000 independent trials and record: node failures, blocks permanently lost (all r replicas failed before re-replication completed), re-replication events, and re-replication bandwidth (mean and peak per hour).

Table 18. DHT churn simulation results ($N = 1,000$, $D = 10$ TB, 1 KB blocks, 1,000 trials, 8,760 hours/trial; seed = 42)

Metric	$r = 1$	$r = 2$	$r = 3$
Analytical failures/year	1,000	1,000	1,000
Sim. failures/year [95% CI]	999 [940, 1,059]	999 [938, 1,063]	1,000 [933, 1,066]
Sim. blocks lost/year (mean)	1.07×10^{10}	579,821	0
Sim. blocks lost [95% CI]	$[1.01, 1.14] \times 10^{10}$	$[0, 10,737,418]$	$[0, 0]$
Sim. data lost/year	~ 10 TB (total)	~ 566 MB	0 B

Metric	r = 1	r = 2	r = 3
Re-rep events/day	—	2.74	2.74
Mean re-rep BW/day [95% CI]	—	28.0 GB [26.3, 29.8]	28.1 GB [26.2, 29.9]
Peak re-rep BW/hour [95% CI]	—	29.8 GB [20.5, 41.0]	30.0 GB [20.5, 41.0]

CI denotes 95% confidence interval across trials. Re-replication bandwidth reflects full partition transfer ($D/N = 10$ GB) per failure event.

The simulated failure rates match the analytical Poisson prediction within the 95% confidence interval across all replication factors. At $r = 3$, no simulation trial recorded any permanent block loss in 1,000 simulated years, consistent with the analytical prediction of ~ 8 blocks/year (an event requiring three near-simultaneous adjacent node failures within the 30-second heartbeat window).

At $r = 2$, the simulated mean block loss (579,821 blocks ≈ 566 MB/year) exceeds the analytical steady-state prediction. This discrepancy arises because the simulation captures correlated failure events (two adjacent nodes failing within the re-replication window) that the independence-based analytical model underestimates. The wide confidence interval [0, 10,737,418] reflects the binary nature of correlated failures: in most trials no correlated failure occurs, but when one does, the entire shared partition is at risk. This finding supports the recommendation that production deployments use $r \geq 3$ for data requiring high durability guarantees.

9.7.3. Challenge-response timing distribution simulation. We simulate the timing of challenge-response rounds to validate the outsourcing-resistance argument (Theorem 3) and the timing-based collusion detection claim (Theorem 7). We model three scenarios with distinct latency distributions:

Honest provider. Local SSD read: $\text{LogNormal}(\mu = \ln(0.5), \sigma = 0.3)$ ms. This models sub-millisecond SSD I/O with log-normal variability reflecting OS scheduling jitter and queue depth effects.

Naive adversary. WAN retrieval: $\text{Normal}(\mu = 50, \sigma = 15)$ ms, floored at 5 ms. This models retrieval from a remote data source over the public internet.

Sophisticated adversary. Co-located cache via LAN: LogNormal($\mu = \ln(3.0)$, $\sigma = 0.4$) ms. This models an adversary who maintains a nearby (same data-centre) cache of outsourced data, minimising retrieval latency. This is the strongest adversary model for timing analysis.

For each scenario, we generate 10,000 response times and compute distributional statistics. We then perform two-sample Kolmogorov–Smirnov (KS) tests at sample sizes $n \in \{5, 10, 20, 50, 100\}$ to determine the minimum number of observed challenge-response epochs required to distinguish honest from adversarial providers with specified confidence. Each KS test is repeated 10,000 times per sample size to estimate statistical power.

Table 19. Challenge-response timing distributions (10,000 samples per scenario; seed = 42)

Statistic	Honest	Naive adversary	Sophisticated adversary
Mean (ms)	0.522	50.31	3.271
Median (ms)	0.498	50.14	3.022
Std dev (ms)	0.161	15.02	1.368
P1 (ms)	0.245	15.75	1.196
P99 (ms)	1.014	84.89	7.826
Separation vs honest	—	96.4×	6.3×

Table 20. KS test statistical power for detecting outsourcing (10,000 repeated tests per configuration; seed = 42)

Epochs (n)	Power @ $\alpha=0.05$ (naive)	Power @ $\alpha=0.01$ (naive)	Power @ $\alpha=0.05$ (soph.)	Power @ $\alpha=0.01$ (soph.)	Power @ $\alpha=0.005$ (soph.)	Median p (soph.)
5	1.000	1.000	0.9965	0.9965	0.000	7.9×10^{-3}
10	1.000	1.000	1.000	1.000	1.000	1.1×10^{-5}
20	1.000	1.000	1.000	1.000	1.000	1.5×10^{-11}
50	1.000	1.000	1.000	1.000	1.000	2.0×10^{-29}
100	1.000	1.000	1.000	1.000	1.000	2.2×10^{-59}

Power is the fraction of tests rejecting H_0 (honest and adversary distributions are identical) at the specified significance level α .

Even against the sophisticated adversary with a co-located cache (6.3× mean separation), the KS test achieves 99.65% power at significance level $\alpha = 0.01$ with only 5 observed epochs, and 100% power at $n = 10$ for $\alpha \geq 0.01$ (99.98% at $\alpha = 0.001$). However, at significance level $\alpha =$

0.005, power drops to zero at $n = 5$ because the discrete KS test at this sample size yields a fixed p-value of approximately 0.008, which falls between the two thresholds. This cliff-edge is an artefact of the small-sample discrete KS distribution and vanishes at $n = 10$, where power reaches 100% at all significance levels tested (including $\alpha = 0.005$ and $\alpha = 0.001$). Practitioners requiring $\alpha < 0.01$ should therefore collect at least 10 challenge epochs. The honest provider's 99th-percentile response time (1.014 ms) is below the sophisticated adversary's 1st-percentile response time (1.196 ms), yielding a gap of 0.182 ms with only 0.32% of sophisticated adversary samples falling below the honest P99 threshold. Against the naive WAN-based adversary (96.4× separation), detection is trivially certain at all tested sample sizes.

Note on timing granularity. The response times in Table 19 are measured at the application layer (mempool arrival time as observed by monitoring nodes), not at block-confirmation granularity. BSV's block interval (~10 minutes) is far too coarse for sub-millisecond discrimination. The timing analysis therefore requires that the oracle or an independent monitoring node record mempool arrival timestamps for response transactions with at least millisecond precision. This is operationally feasible: Bitcoin node implementations record mempool arrival times in their logs, and multiple independent monitors can cross-validate timestamps to prevent manipulation. The protocol specification mandates that challenge-response timing is measured at mempool arrival, not block inclusion.

Limitation: adversary timing manipulation. A sophisticated adversary aware of the timing analysis could artificially delay some responses to approximate the honest distribution. However, this strategy is self-defeating: delaying responses increases the risk of missing the challenge deadline τ , triggering slashing (Section 9.3). An adversary who delays $d\%$ of responses to mimic honest timing forfeits $d\%$ of expected revenue. The timing analysis therefore operates in conjunction with the economic mechanism, not independently.

9.7.4. Simulation limitations and scope. The simulations model idealised conditions: independent node failures (Poisson process), uniform random challenge selection, fixed network latency distributions, and instantaneous failure detection (bounded only by the heartbeat interval). Production deployments may exhibit: (i) correlated failures from shared infrastructure (e.g., data-centre power events, network partitions); (ii) non-uniform latency distributions reflecting geographic dispersion; (iii) adversarial timing manipulation as discussed above; and (iv) DHT routing delays not captured in the point-to-point latency model.

The purpose of the simulations is to validate internal consistency between the analytical models (Theorems 2–4, 7) and Monte Carlo execution under their stated assumptions, not to substitute for empirical measurement on a deployed prototype. The simulation code, parameter files, and complete output logs are provided as Supplementary Material to enable independent verification and extension.

10. Discussion

10.1. Comparison with Existing Systems

Table 21. Comparison with existing decentralised storage systems

Feature	Proposed	Filecoin	Storj	IPFS	Sia
Proof type	Merkle+script	zk-SNARK PoSt	Audit (satellite)	None	Merkle (custom)
Custom blockchain	No	Yes	No	N/A	Yes
Public verifiability	Yes (on-chain)	Yes (on-chain)	No (satellite)	No	Yes (on-chain)
Hardware requirement	Standard	GPU (sealing)	Standard	Standard	Standard
Collusion resistance	Per-copy encrypt	PoRep encoding	Erasur coding	None	None
DHT partitioning	Yes (Pastry)	No	Yes	Yes (Kademlia)	No
Trust assumption	Blockchain only	Blockchain	Satellite	Content addr.	Blockchain

The proposed architecture occupies a distinct position in the design space. Unlike Filecoin, it requires no custom blockchain, no GPU-intensive sealing, and no zk-SNARK generation—operating instead on any UTXO-based blockchain with standard scripting. Unlike Storj, it eliminates centralised satellite coordinators by embedding verification in the blockchain’s transaction validation pipeline. Unlike IPFS, it provides native verification of continued storage, not merely content addressing. Unlike Sia, it operates on a general-purpose blockchain rather than requiring a dedicated chain and cryptocurrency. As [10] note in their comprehensive survey, the trade-off between verification cost and decentralisation remains the central design challenge in blockchain-based storage—the present architecture resolves this by leveraging existing blockchain infrastructure rather than building new.

10.2. Economic Model

The economic structure embeds payment in the verification mechanism itself. The output of each challenge transaction carries the storage fee; unlocking it requires a valid proof. This

creates an atomic verify-then-pay mechanism: the storage provider is compensated if and only if it demonstrates continued possession. No external enforcement (legal contracts, reputation systems) is needed beyond the blockchain's own consensus. The deposit and slashing schedule (Section 9.3) ensures that this incentive alignment operates over the full contract duration, not merely at individual challenge epochs. Under the parameterisation in Section 9.2, a rational storage provider maximises expected revenue by retaining data, since the marginal cost of disk storage is lower than the expected forfeited deposit from failing challenges. The systems evaluation (Section 9.2) quantifies these costs: at the large deployment scale (1 PB, 1,000 providers), total annual verification cost is \$336—approximately 0.12% of equivalent AWS S3 Standard pricing at published rates [36]—indicating that the economic overhead of on-chain verification is small relative to storage costs.

10.3. Scalability

The on-chain footprint is modest: ~200 transactions/day for 100 providers challenged daily. Each challenge-response cycle produces two transactions totalling approximately 1–1.5 KB. Off-chain overhead per node includes the DHT routing table ($\sim O(\log N)$ entries), Merkle tree metadata ($\sim 0.2\%$ of stored data volume), and the reputation table for the local neighbourhood. The architecture scales horizontally: adding storage nodes increases total capacity proportionally while maintaining per-node storage and communication costs constant, a property that distinguishes it from full-replication architectures where per-node cost grows with network size.

10.4. Applications

IoT telemetry. Sensor data distributed across storage nodes via DHT; periodic blockchain challenges verify compliance-critical records. Each challenge provides a publicly auditable timestamp proving data existed at a specific time—satisfying regulatory requirements for IoT data provenance without centralised infrastructure. [23] have shown that sampling-based verification can reduce IoT data integrity overhead while maintaining high confidence levels, a result directly applicable to the periodic challenge schedule proposed here.

Enterprise archival. Organisations required to retain financial records for 7–30 years receive cryptographic proof of continuous retention, satisfying regulatory requirements (e.g., SOX, GDPR data retention obligations) without relying on the storage provider's self-attestation. The blockchain provides an immutable audit trail of all verification events.

Content delivery. Encrypted copies distributed to providers worldwide; RAID striping enables parallel delivery; per-copy encryption prevents unauthorised redistribution while maintaining verifiable storage guarantees.

Supply chain provenance. Documentation (inspection reports, certifications, shipping manifests) stored in the DHT network with blockchain verification provides tamper-proof provenance chains that persist independently of any single supply chain participant.

10.5. Limitations and Future Work

The architecture has several limitations that define directions for future work. First, proof-of-retention verifies possession at discrete challenge points, not continuously. Between challenges, a storage provider could temporarily delete data and re-acquire it. While the encrypted multi-copy scheme makes this re-acquisition infeasible (Theorem 3), the underlying limitation remains: our protocol establishes proof-of-possession-at-challenge-time, with retention inferred probabilistically across epochs (Theorem 4). Achieving continuous retention guarantees would require hardware-based attestation (e.g., trusted execution environments), which is outside the scope of this work. Second, the reputation system parameters (initial score, adjustment rates, eviction threshold) require empirical tuning through deployment experience. The current analytical model demonstrates the correct scaling behaviour, but concrete parameter values must be calibrated to specific network conditions. Third, the service provider represents a point of partial centralisation. While challenge issuance is verifiable (Section 8.5) and Merkle roots are immutable once committed, the service provider still performs data preparation and key management. Future work should explore decentralised oracle protocols using threshold cryptography (e.g., t-of-m key shares for encryption key management) or smart contract automation of challenge scheduling. Fourth, the systems evaluation presented here is analytical rather than empirical; a deployed prototype with measured throughput, latency, and failure recovery under realistic workloads remains necessary for production readiness.

Additional directions include game-theoretic analysis of economic incentives including Nash equilibrium of race-based proofs; extension to dynamic data with Merkle tree updates as studied by [21] using hierarchical structures; integration with SNARKs for proof-size compression in bandwidth-constrained environments; and cross-chain compatibility for geographic and jurisdictional diversity in storage provider selection.

11. Conclusion

This paper has proposed a blockchain-verified distributed storage architecture combining DHT-based data partitioning with Merkle proof-of-retention enforced through blockchain transaction scripting. The architecture comprises three integrated mechanisms: a DHT overlay network partitioning data across storage nodes with $O(\log N)$ routing, reducing per-node storage by N/r compared to full replication; a Merkle proof-of-retention protocol enabling compact $O(\log n)$ byte cryptographic verification embedded directly in blockchain transactions; and an encrypted multi-copy scheme with formally proven collusion resistance under standard cryptographic assumptions.

We have demonstrated implementability through a concrete BSV script specification using only five opcode types (OP_SHA256, OP_CAT, OP_SWAP, OP_NOTIF, OP_EQUAL), requiring exactly $6d + 3$ opcodes for a Merkle tree of depth d . The protocol is portable to any UTXO chain supporting these operations, with BSV's removed script limits providing the most permissive execution environment. Three privacy-preserving proof variants (ciphertext-only, hash-only, and EC-point) ensure that no plaintext data is exposed on-chain. The EC-point variant is supported by formal security reductions to ECDLP hardness and ECDSA unforgeability (Theorems 8–10).

Formal security analysis establishes results through reduction-style proofs across three oracle trust models (semi-honest, malicious, and colluding): (1) possession soundness reduces to collision resistance of SHA-256, with adversarial success probability bounded by 2^{-127} ; (2) outsourcing resistance is guaranteed under the encrypted multi-copy scheme, where each provider's ciphertext is unique and unavailable from any external source; (3) selective-retention detection is quantified, showing that a storage provider deleting 1% of blocks is detected with 99% probability within 461 daily challenges; and (4) oracle-provider collusion, while not cryptographically preventable under a single-oracle architecture, is publicly detectable through statistical timing analysis and structurally mitigable through threshold oracle decentralisation. The DHT layer is secured against eclipse and churn manipulation attacks through VRF-based identifier assignment, reputation-based admission, and economic deposit requirements.

Systems evaluation demonstrates economic viability: verification costs range from \$2.44/year for a 100 GB deployment to \$336/year for a 1 PB deployment at BSV's current fee rates, representing less than 0.13% of equivalent cloud storage costs. Fee sensitivity analysis confirms robustness to more than two orders of magnitude increase in fee rates before

economic parity with cloud storage is reached. A structured deposit-and-payment schedule with slashing conditions prevents single-epoch-and-disappear behaviour. Monte Carlo simulation (10,000 trials per configuration, seed = 42) validates the analytical models: selective-retention detection times match theoretical predictions within 2.2% for most deletion fractions (and within 6.7% at $\alpha = 0.10$ where discrete-distribution effects dominate), DHT churn dynamics are confirmed with corrected re-replication bandwidth estimates, and timing-based outsourcing detection achieves 99.65% statistical power at significance level 0.01 with as few as 5 observed challenge epochs even against a sophisticated adversary with co-located cache infrastructure (reaching $\geq 99.98\%$ power at all tested significance levels including $\alpha = 0.001$ with 10 epochs).

The design resolves the fundamental verification problem in remote data storage by combining the mathematical guarantees of Merkle trees with the economic enforceability and public auditability of blockchain transactions. The result is a storage verification system where storage claims are cryptographically verifiable rather than contractually assumed, and where verification is publicly auditable rather than privately administered—establishing a foundation for trustless distributed storage at scale.

Conflict of Interest

The author declares no conflict of interest.

Acknowledgements

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data Availability

The simulation code, parameter files, and complete output logs supporting the results in Section 9.7 are provided as Supplementary Material (supplementary_{simulations}.zip). All simulations are deterministic (NumPy seed = 42) and fully reproducible with Python ≥ 3.9 , NumPy ≥ 1.21 , and SciPy ≥ 1.7 . No empirical datasets were used; all results are analytically derived or computationally generated from the specified parameters.

Data and Code Availability

All simulation code, reference outputs, and a verification script are provided as supplementary material accompanying this submission. The replication package includes requirements.txt for

environment pinning and `verify_outputs.py` for automated reproducibility checking. Running `python verify_outputs.py` regenerates all simulation results and confirms exact agreement with the bundled reference outputs (seed = 42).

References

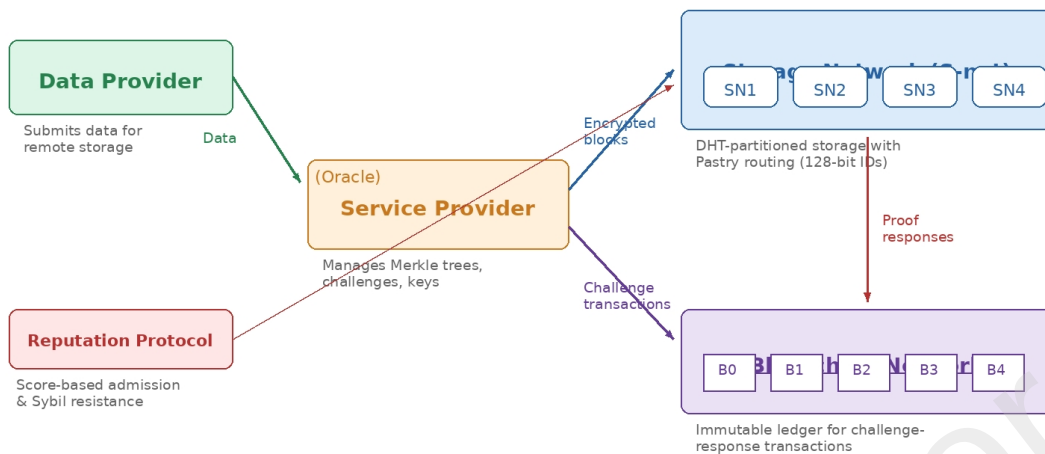
- [1] Statista, Volume of data created worldwide 2010–2025, 2023. URL: <https://www.statista.com/statistics/871513/> (accessed 19 February 2026).
- [2] D. Ford, et al., Availability in globally distributed storage systems, in: Proc. 9th USENIX OSDI, 2010, pp. 61–74.
- [3] B. Schroeder, R. Lagisetty, A. Merchant, Flash reliability in production, in: Proc. 14th USENIX FAST, 2016, pp. 67–80.
- [4] T. Li, J. Chu, L. Hu, CIA: A collaborative integrity auditing scheme for cloud data with multi-replica on multi-cloud storage providers, *IEEE Trans. Parallel Distrib. Syst.* 34 (1) (2023) 154–162.
- [5] H. Chen, Y. Lu, Y. Cheng, FileInsurer: A scalable and reliable protocol for decentralised file storage in blockchain, in: Proc. IEEE 42nd ICDCS, 2022, pp. 168–179.
- [6] J. Benet, IPFS – Content addressed, versioned, P2P file system, Preprint, arXiv:1407.3561, 2014.
- [7] Protocol Labs, Filecoin: A decentralized storage network, White Paper, 2017. URL: <https://filecoin.io/filecoin.pdf> (accessed 19 February 2026).
- [8] S. Williams, et al., Arweave: A protocol for economically sustainable information permanence, White Paper, 2019.
- [9] S. Wilkinson, T. Boshevski, J. Brandoff, V. Buterin, Storj: A peer-to-peer cloud storage network, White Paper, 2014.
- [10] M.I. Khalid, I. Ehsan, A.K.I. Al-Ani, J. Iqbal, S. Hussain, S.S. Ullah, Nayab, A comprehensive survey on blockchain-based decentralized storage networks, *IEEE Access* 11 (2023) 10995–11015.
- [11] S. Ghemawat, H. Gobioff, S.-T. Leung, The Google file system, in: Proc. 19th ACM SOSP, 2003, pp. 29–43.

- [12] K. Shvachko, H. Kuang, S. Radia, R. Chansler, The Hadoop distributed file system, in: Proc. 26th IEEE MSST, 2010, pp. 1–10.
- [13] G. DeCandia, et al., Dynamo: Amazon’s highly available key-value store, in: Proc. 21st ACM SOSP, 2007, pp. 205–220.
- [14] J. Kubiawicz, et al., OceanStore: An architecture for global-scale persistent storage, in: Proc. 9th ACM ASPLOS, 2000, pp. 190–201.
- [15] F. Dabek, M.F. Kaashoek, D. Karger, R. Morris, I. Stoica, Wide-area cooperative storage with CFS, in: Proc. 18th ACM SOSP, 2001, pp. 202–215.
- [16] R. Nanda, S. Srivastava, S. Jain, V. Yadav, A. Sharma, Decentralised storage framework using Hyperledger Fabric and IPFS, in: Proc. 2024 ICCS, 2024.
- [17] G. Ateniese, R. Burns, R. Curtmola, J. Herring, L. Kissner, Z. Peterson, D. Song, Provable data possession at untrusted stores, in: Proc. 14th ACM CCS, 2007, pp. 598–609.
- [18] A. Juels, B.S. Kaliski, PORs: Proofs of retrievability for large files, in: Proc. 14th ACM CCS, 2007, pp. 584–597.
- [19] H. Shacham, B. Waters, Compact proofs of retrievability, in: Proc. ASIACRYPT 2008, 2008, pp. 90–107.
- [20] C. Wang, S.S.M. Chow, Q. Wang, K. Ren, W. Lou, Privacy-preserving public auditing for secure cloud storage, IEEE Trans. Comput. 62 (2) (2011) 362–375.
- [21] Z. Liu, S. Wang, S. Duan, L. Ren, J. Wei, Dynamic data integrity auditing based on hierarchical Merkle hash tree in cloud storage, Electronics 12 (3) (2023) 717.
- [22] Z. Liu, et al., Data integrity audit scheme based on quad Merkle tree and blockchain, IEEE Access 11 (2023) 10026822.
- [23] H. Tang, et al., Study on data storage and verification methods based on improved Merkle mountain range in IoT scenarios, J. King Saud Univ. Comput. Inf. Sci. 36 (7) (2024) 102117.
- [24] O. Kuznetsov, A. Rusnak, A. Yezhov, K. Kuznetsova, D. Kanonik, O. Domin, Evaluating the security of Merkle trees: An analysis of data falsification probabilities, Cryptography 8 (3) (2024) 33.

- [25] Filecoin Docs, Storage proving: WindowPoSt challenges, Technical Documentation, 2023. URL: <https://docs.filecoin.io/storage-providers/filecoin-economics/storage-proving> (accessed 19 February 2026).
- [26] D. Vorick, L. Champine, Sia: Simple decentralized storage, White Paper, 2014. URL: <https://sia.tech/sia.pdf> (accessed 19 February 2026).
- [27] I. Stoica, R. Morris, D. Karger, M.F. Kaashoek, H. Balakrishnan, Chord: A scalable P2P lookup service, ACM SIGCOMM CCR 31 (4) (2001) 149–160.
- [28] A. Rowstron, P. Druschel, Pastry: Scalable, decentralized object location and routing, in: Middleware 2001, 2001, pp. 329–350.
- [29] P. Maymounkov, D. Mazières, Kademlia: A peer-to-peer information system based on the XOR metric, in: IPTPS, 2002, pp. 53–65.
- [30] S. Ratnasamy, P. Francis, M. Handley, R. Karp, S. Shenker, A scalable content-addressable network, in: Proc. ACM SIGCOMM, 2001, pp. 161–172.
- [31] C. Lesniewski-Laas, M.F. Kaashoek, Whanau: A Sybil-proof distributed hash table, in: Proc. 7th USENIX NSDI, 2010, pp. 1–16.
- [32] N. Borisov, Computational puzzles as Sybil defenses, in: Proc. 6th IEEE P2P, 2006, pp. 171–176.
- [33] S.D. Kamvar, M.T. Schlosser, H. Garcia-Molina, The EigenTrust algorithm for reputation management in P2P networks, in: Proc. 12th WWW, 2003, pp. 640–651.
- [34] Q. Gan, et al., Certificateless data integrity auditing with sparse Merkle trees for the cloud-edge environment, Sci. Rep. 14 (2024) 14041.
- [35] J. Poon, T. Dryja, The Bitcoin Lightning Network: Scalable off-chain instant payments, White Paper, 2016.
- [36] Amazon Web Services, Amazon S3 pricing, 2024. URL: <https://aws.amazon.com/s3/pricing/> (accessed 19 February 2026).
- [37] Bitcoin SV Node Team, Genesis upgrade: Restoring the original Bitcoin protocol, Bitcoin SV Technical Standards Committee, 2020. URL: <https://bitcoinsv.io/genesis-hard-fork/> (accessed 19 February 2026).

[38] S. Nakamoto, Bitcoin: A peer-to-peer electronic cash system, 2008. URL: <https://bitcoin.org/bitcoin.pdf> (accessed 19 February 2026).

System Architecture Overview



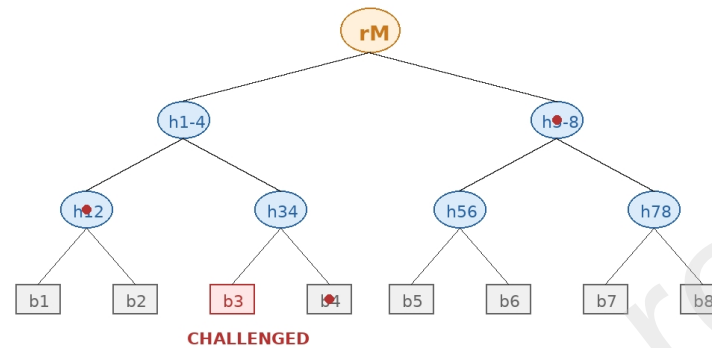
Key flows:

- Data submission
- Encrypted block distribution
- Challenge / Proof via blockchain

Merkle Proof-of-Retention Protocol

Challenge-Response Flow:

- 1 Oracle selects random leaf index i
- 2 Constructs Tchal with locking script embedding rM and leaf index
- 3 Storage provider retrieves block b_i and Merkle siblings
- 4 Constructs Tresp with unlocking script containing b_i + proof path
- 5 Every miner verifies:
 $H(H(\dots)||\text{sibling}) \dots = rM$



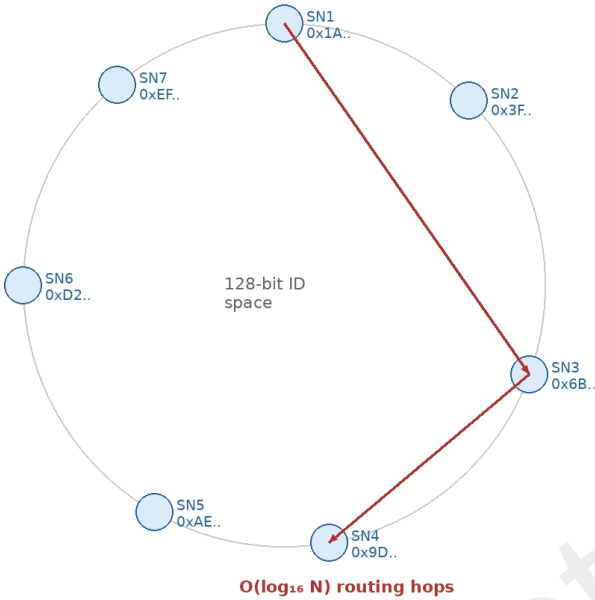
Locking Script (in Tchal):

OP_SHA256 OP_CAT OP_SHA256
... (d rounds) ...
OP_EQUALVERIFY < rM >

Unlocking Script (in Tresp):

< b_i > <sibling_d> ... <sibling_1>
(leaf preimage + Merkle path)

DHT-Based Data Distribution



Per-Node Data Structures

Routing Table

Prefix ID Node address
(long distance forwarding)

Neighbourhood Set

Nearest peers by latency
(local routing optimisation)

Leaf Set

1/2 closest IDs each side
(keyspace neighbour routing)

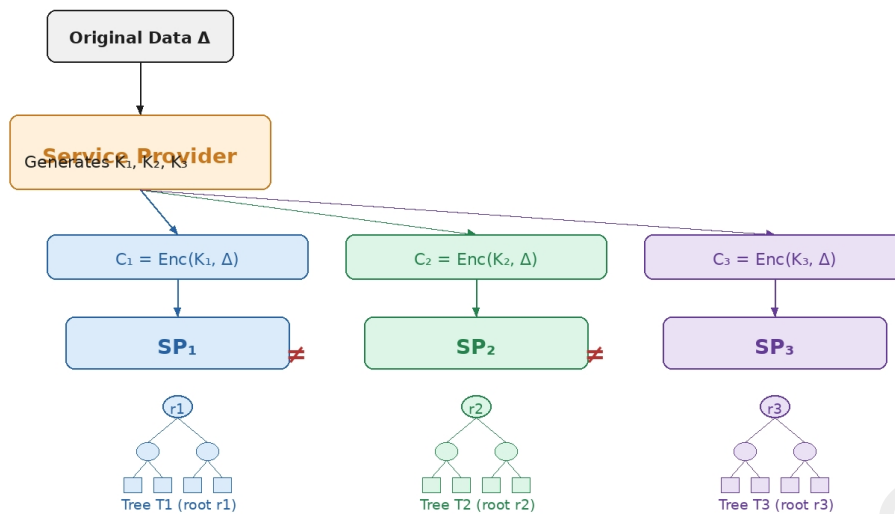
Assigned Data Partition

rB DNs per node
(replication factor $r = 2$)

Data block assignment:

$k = H(\text{block_id}) \rightarrow \text{DHT route} \rightarrow \text{put}(k, \text{block})$

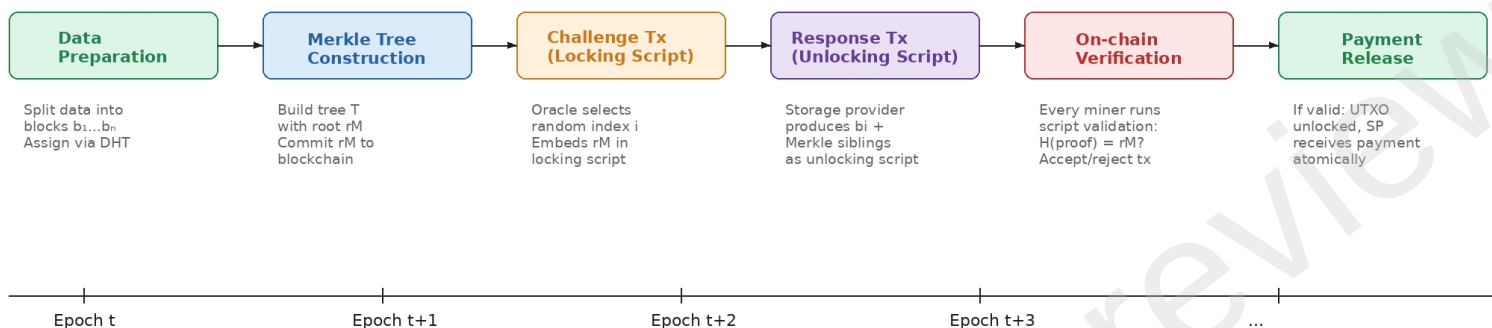
Encrypted Multi-Copy Collusion Resistance



Collusion Prevention:

Each SP holds unique ciphertext $\rightarrow$ different Merkle trees $\rightarrow$ proofs for SP_1 cannot satisfy challenges for SP_2 .
Forging cross-copy proofs requires breaking IND-CPA encryption or SHA-256 collision resistance

Challenge-Response Transaction Lifecycle



Periodic challenges ensure probabilistic full coverage:

After k epochs: $P(\text{block } i \text{ challenged} \geq 1) = 1 - (1 - 1/n)^k$

For $n=1000$, $k=7000$: $P > 0.999$

Confirmation tracking: counter increments per block; prune after 4-7 confirms